

BỘ TÀI LIỆU KỸ THUẬT · TÀI LIỆU 02

DAAB

Kiến trúc hệ thống

Xương sống dữ liệu & tri thức: một nền tảng knowledge graph kết nối với các cơ sở dữ liệu đang vận hành và kho tài liệu của một tổ chức, rồi cung cấp chúng cho AI agent qua Model Context Protocol (MCP) và cho con người qua một web dashboard.



PHIÊN BẢN
1.0

NGÀY
2026-09-24

PHIÊN BẢN MÃ NGUỒN
go @ 16dc548 · python @ 6966d4b · web @
907bc8b

Mục lục

1 Tổng quan & nguyên tắc thiết kế

Nền tảng làm được gì · Nguyên tắc thiết kế · Công nghệ sử dụng

2 Bối cảnh hệ thống

Các bên bên ngoài · Dữ liệu đi đâu

3 Container & thành phần

Góc nhìn container · Bên trong Go API server

4 Mô hình knowledge graph

Các loại node · Cạnh và whitelist · Cổng ghi và lịch sử

5 Mô hình dữ liệu

Lỗi graph · Truy cập · Nguồn dữ liệu · Truy vấn và ghi · Tài liệu · Hợp nhất và bộ nhớ

6 Pipeline nạp dữ liệu

Tài liệu · Cơ sở dữ liệu đang vận hành · Mã nguồn

7 Truy xuất: tìm kiếm, xếp hạng lại & GraphRAG

Chế độ tìm kiếm · Điều hướng graph · GraphRAG · Chỉ mục vector

8 Truy vấn dữ liệu bằng ngôn ngữ tự nhiên

Pipeline truy vấn của agent · Guard khi thực thi · Phân tích hội thoại trên dashboard

9 Ghi có kiểm soát

Chèn một dòng · Playbook ghi

10 Hợp nhất thực thể & bản ghi chủ

Trích xuất · Các giai đoạn hợp nhất · Áp dụng gộp · Bản ghi chủ và bản ghi dẫn xuất

11 Bộ nhớ của agent

Ghi bộ nhớ · Truy hồi lại · Bảo trì

12 MCP bridge

Phương thức truyền tải · Chế độ xác thực · Pipeline mỗi lượt gọi · Danh mục tool

13 Web dashboard

Sơ đồ trang · BFF proxy và phiên · Thời gian thực · Trực quan hóa graph

14 Giao tiếp giữa các dịch vụ

Danh sách job · Lượt gọi đồng bộ · Ngữ nghĩa phân phối

A Phụ lục A – Danh mục tool MCP

Mọi tool kèm lớp và mục đích

B Phụ lục B – Tóm tắt REST API

Số route theo khu vực · Endpoint tiêu biểu

Thang trạng thái dùng trong toàn tài liệu: **LIVE** đã hiện thực, đã tích hợp và đã chạy với dữ liệu thật · **BUILT** đã hiện thực và đã tích hợp, chưa chạy với dữ liệu production · **PLANNED** nằm trong lộ trình, đang tắt bằng cờ tính năng hoặc mới ở dạng stub. Mọi số liệu trong tài liệu này được đo trên phiên bản mã nguồn in ở trang bìa.

1 Tổng quan & nguyên tắc thiết kế

DAAB biến các cơ sở dữ liệu nghiệp vụ và kho tài liệu của một tổ chức thành một knowledge graph duy nhất có thể truy vấn. AI agent truy cập qua một MCP server theo chuẩn (công bố 55 tool, trong đó 53 tool đã nối API); con người truy cập qua web dashboard. Bên dưới là một hệ thống lưu trữ gốc PostgreSQL duy nhất, chứa đồng thời graph, lịch sử phiên bản, chỉ mục toàn văn và vector embedding.

251 ROUTE HTTP API	53 TOOL MCP ĐÃ NỐI API	86 MIGRATION SCHEMA · 54 BẢNG	22 LOẠI NODE CÓ KIỂU
59 QUY TẮC WHITELIST CẠNH · 26 NHÃN	3,668 HÀM TEST (GO + PYTHON)	4 LOẠI CSDL NGUỒN	7 ĐỊNH DẠNG TÀI LIỆU ĐƯỢC NẠP

Phương pháp: route được đếm từ các pattern `HandleFunc` đã đăng ký (247 trong package handler + 4 trên mux ngoài cùng); tool MCP = 55 schema được công bố trừ 2 schema chưa có route phía server; bảng là các tên `CREATE TABLE` riêng biệt trong các migration 000001–000087, đối chiếu với một cơ sở dữ liệu đã chạy migration; test = 2,753 hàm Go `func Test*` + 915 hàm Python `def test_*` (pytest thu thập 943 test Python, tính cả các trường hợp tham số hóa). Loại CSDL nguồn: PostgreSQL, MySQL, MariaDB, Microsoft SQL Server. Định dạng: PDF, DOCX, Markdown, TXT, CSV, XLSX, JSON.

• Nền tảng làm được gì

- **Kết nối với cơ sở dữ liệu đang vận hành.** Đăng ký các nguồn PostgreSQL, MySQL, MariaDB và SQL Server của khách hàng, trích xuất schema, rồi sinh ra một graph gồm bảng, khóa ngoại và các quan hệ ngầm do AI phát hiện.
- **Nạp tài liệu.** Phân tích cú pháp, OCR (kể cả tiếng Việt), khử trùng lặp, chia mục, chia chunk và tạo embedding cho tài liệu, sau đó trích xuất thực thể và quan hệ vào cùng graph đó.
- **Trả lời câu hỏi trên dữ liệu.** Với agent, chuyển một câu hỏi ngôn ngữ tự nhiên thành một *query plan* đã được kiểm tra, sinh SQL tham số hóa theo cách tắt định và chạy ở chế độ chỉ đọc trên nguồn; chat trên dashboard bổ sung một chế độ hội thoại (Chương 8).
- **Truy xuất tri thức.** Cung cấp tìm kiếm toàn văn, ngữ nghĩa và lai (Reciprocal Rank Fusion), truy xuất ở mức đoạn văn kèm trích dẫn, duyệt nhiều bước và GraphRAG xuyên tài liệu.
- **Hợp nhất thực thể.** Gộp các bản trùng của người, tổ chức và khái niệm qua một pipeline nhiều giai đoạn, có ngưỡng độ tin cậy và có thể đảo ngược.
- **Ghi nhớ.** Cung cấp cho agent một bộ nhớ bền vững, có phạm vi (`remember` / `recall`) với truy hồi lai.
- **Ghi ngược lại, có kiểm soát.** Chèn một dòng khi được bật tường minh và các playbook ghi nhiều bước đã được phê duyệt trước, tất cả đều được ghi nhật ký audit.

• Nguyên tắc thiết kế

Nguyên tắc	Thể hiện trong mã nguồn
Một hệ thống lưu trữ gốc có giao dịch	Node, cạnh, lịch sử phiên bản, vector toàn văn (<code>tsvector</code>) và embedding (<code>pgvector</code>) của graph đều nằm trong PostgreSQL 16. Không có cơ sở dữ liệu graph hay vector riêng cần đồng bộ; một lần ghi và cập nhật chỉ mục của nó được commit cùng nhau.

Nguyên tắc	Thể hiện trong mã nguồn
Một đường ghi duy nhất	Go API server sở hữu cơ sở dữ liệu. Thao tác ghi vào graph từ worker Python, dashboard và MCP bridge đều đi qua REST API của nó, nên kiểm tra hợp lệ, phân quyền và audit áp dụng cho mọi bên ghi. Ngoại lệ duy nhất là một kết nối cơ sở dữ liệu trực tiếp tùy chọn cho worker (được đặt trong stack phát triển), dùng để đánh dấu đề xuất gộp cần duyệt và tra cứu tài liệu.
Schema là cấu hình	Loại node, thuộc tính bắt buộc, quy tắc đầy đủ và whitelist cạnh được khai báo trong <code>config.yaml</code> và được thực thi bởi các cổng kiểm tra. Loại tri thức mới chỉ cần cấu hình, không cần bảng mới.
Lịch sử có phiên bản	Mỗi lần cập nhật node được một trigger cơ sở dữ liệu sao chép vào bảng phiên bản, và thao tác sửa graph không bao giờ xóa phiên bản. Node tri thức được đánh dấu lỗi thời (deprecated) hoặc bị thay thế (superseded) thay vì bị xóa; ngoại lệ là việc sinh lại tài liệu, thao tác này thay thế các mục và chunk của tài liệu đã thay đổi (cùng các dòng phiên bản của chúng). Cập nhật dùng khóa lạc quan (optimistic locking).
Mô hình đề xuất, mã nguồn quyết định	Với truy vấn dữ liệu của agent (MCP / API), LLM trả về một query plan có cấu trúc, không bao giờ là SQL thô; chế độ agentic của chat trên dashboard cho phép mô hình tự viết câu lệnh SELECT, được kiểm tra bởi một bộ kiểm tra chỉ đọc có giới hạn số dòng. Quyết định gộp phải qua các cổng quy tắc, reranker và độ tin cậy. Playbook ghi được đề xuất theo cách tắt định và phải được phê duyệt trước khi chạy.
Embedding cục bộ	Embedding (multilingual-e5-small, 384 chiều) và reranker (bge-reranker-v2-m3) chạy trên CPU bên trong nền tảng. Tìm kiếm và hợp nhất không tốn chi phí embedding theo lượt gọi, và nội dung tài liệu không bị gửi ra ngoài để tạo embedding (một dịch vụ embedding được lưu trữ bên ngoài, tùy chọn, chỉ dùng để chọn bảng khi được cấu hình).
AI không phụ thuộc nhà cung cấp	Nhà cung cấp LLM được đăng ký trong cơ sở dữ liệu với khóa đã mã hóa, được chọn theo độ ưu tiên, có tự động chuyển đổi dự phòng (failover) và circuit breaker. Mô hình được gán theo từng chức năng (ví dụ lập query plan khác với trích xuất). Chat trên dashboard và, khi được cấu hình, trích xuất ở mức chunk gọi nhà cung cấp trực tiếp từ các dịch vụ Python (Chương 3).
Ưu tiên thư viện chuẩn	Go server dùng <code>net/http</code> và <code>database/sql</code> với SQL viết tay, không dùng web framework, không dùng ORM. Nhờ đó bề mặt phụ thuộc nhỏ và truy vấn dễ dự đoán.

Trạng thái triển khai: giai đoạn tiền production (chưa triển khai production)

Hiện DAAB chạy dưới dạng stack Docker Compose tự host cho phát triển và trình diễn **LIVE**. Một bản phát hành (v1.0.0) và một môi trường cloud dev trên AWS (ECS, RDS, ElastiCache) có CI/CD cho dịch vụ Go đã được xây dựng **BUILT**; triển khai production ở trạng thái **PLANNED**. Tài liệu này không khẳng định có khách hàng production, SLA hay số liệu quy mô. Hạ tầng, bảo mật và vận hành được trình bày trong Tài liệu 03.

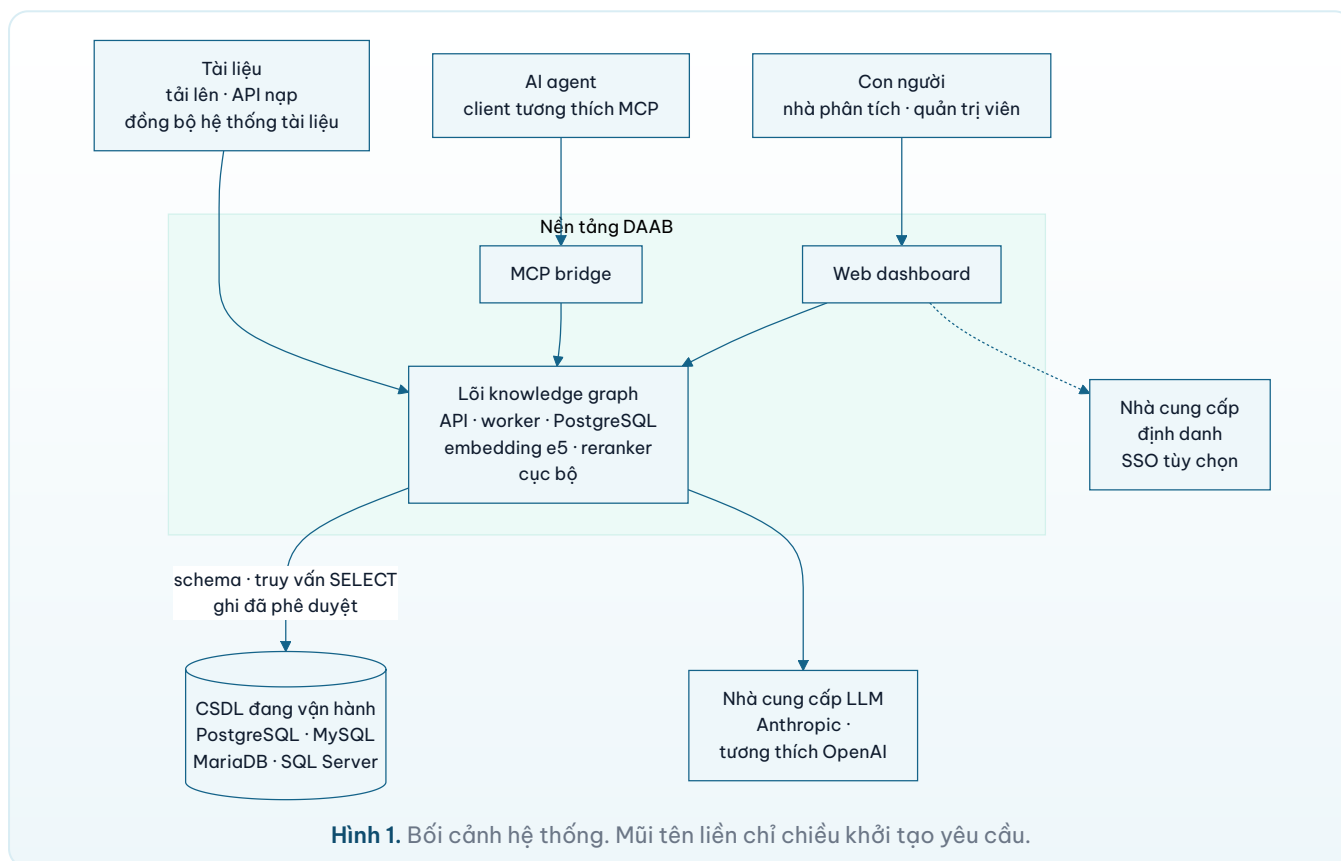
• Công nghệ sử dụng

Tầng	Công nghệ
API server & MCP bridge	Go 1.25, stdlib <code>net/http</code> (pattern theo method của 1.22), <code>database/sql</code> + <code>lib/pq</code> , golang-migrate, MCP Go SDK chính thức, <code>log/slog</code> , AES-256-GCM

Tầng	Công nghệ
Worker & dịch vụ AI	Python 3.12, FastAPI + uvicorn, httpx, pydantic-settings, sentence-transformers (CPU), tree-sitter, Tesseract + RapidOCR, pypdf/pypdfium2, asyncpg / pymysql / pymssql
Web dashboard	Next.js 16 (App Router), React 19, TypeScript strict, TanStack Query 5, iron-session, Tailwind 4, Three.js / React Three Fiber, Cytoscape
Dữ liệu	PostgreSQL 16 với pgvector , pg_trgm , unaccent ; Redis 7 (danh sách job, trạng thái agent ngắn hạn)
Nhà cung cấp AI	Anthropic API, gói thuê bao Claude qua OAuth, mọi endpoint tương thích OpenAI; embedding e5 và reranker bge chạy cục bộ

2 Bối cảnh hệ thống

DAAB nằm giữa dữ liệu của một tổ chức (cơ sở dữ liệu và tài liệu) và những bên sử dụng tri thức đó: con người trên trình duyệt và AI agent giao tiếp bằng MCP.



Dữ liệu đi đâu

Embedding và rerank chạy cục bộ. Văn bản chỉ rời khỏi hệ thống triển khai khi được gửi tới nhà cung cấp LLM do quản trị viên cấu hình, hoặc tới một dịch vụ embedding bên ngoài (tùy chọn) để chọn bảng, khi được cấu hình. Thông tin xác thực của CSDL nguồn và API key của nhà cung cấp được lưu ở dạng mã hóa (AES-256-GCM) và chỉ được giải mã khi sử dụng. Với chat trên dashboard, Go server chuyển API key của nhà cung cấp và chuỗi kết nối còn ở dạng mã hóa tới Python indexer trong mỗi yêu cầu; indexer tự giải mã chuỗi kết nối và truy vấn trực tiếp cơ sở dữ liệu nguồn (Chương 8).

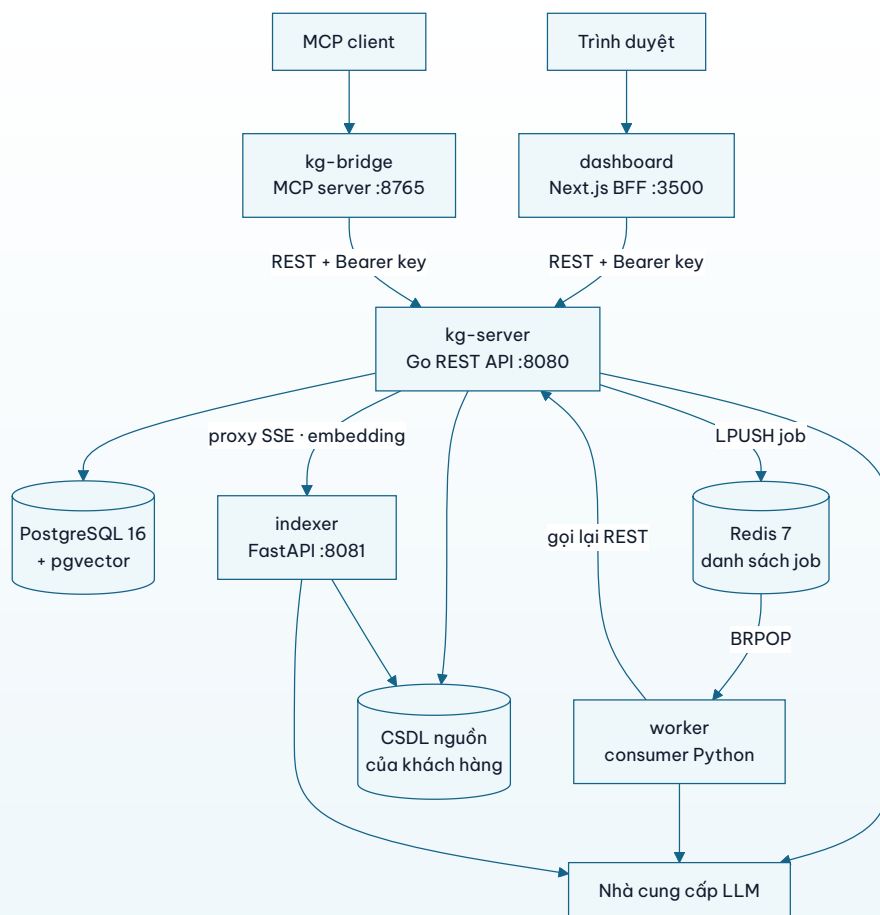
Bên ngoài	Giao diện	Nội dung trao đổi
Con người	Web dashboard (HTTPS)	Khám phá graph, phân tích dữ liệu dạng hội thoại, quản lý nguồn, duyệt dữ liệu nạp, quản trị.
AI agent	MCP qua stdio hoặc Streamable HTTP	53 tool đã nối API: tìm kiếm, duyệt graph, điều hướng tài liệu, truy vấn dữ liệu, bộ nhớ, nạp dữ liệu, ghi có kiểm soát.
CSDL đang vận hành	Driver gốc cho từng loại CSDL	Trích xuất danh mục schema; thực thi truy vấn chỉ SELECT; thao tác chèn và giao dịch playbook được bật tường minh và có audit.
Nguồn tài liệu	Tải tệp lên, API nạp công khai, đồng bộ với hệ thống quản lý tài liệu (cấu hình như một kết nối nguồn)	Tệp và bản nháp có cấu trúc, được chuyển thành tài liệu, mục, chunk và thực thể.
Nhà cung cấp LLM	HTTPS (API của nhà cung cấp)	Lập query plan, trích xuất có cấu trúc, phát hiện quan hệ, xác minh gộp, soạn câu trả lời.
Nhà cung cấp định danh	Xác minh JWT (JWKS)	Đăng nhập một lần (SSO) tùy chọn. Một lần đăng nhập đã xác minh được đổi lấy một DAAB API key.

3 Container & thành phần

Bảy container chạy từ ba image: một image Go (API server và MCP bridge), một image Python (dịch vụ indexer và worker) và một image Next.js (dashboard), cộng thêm PostgreSQL và Redis.

Container	Công nghệ	Trách nhiệm
kg-server	Go, stdlib HTTP	Hệ thống lưu trữ gốc và API gateway. Sở hữu graph, các cổng kiểm tra, xác thực/phân quyền, RBAC theo dự án, audit. Chứa pipeline truy vấn dữ liệu bằng ngôn ngữ tự nhiên, trích xuất schema CSDL đang vận hành và sinh graph, đường ghi, tầng nhà cung cấp AI, xây dựng liên kết GraphRAG và job engine chạy trong tiến trình.
kg-bridge	Go, MCP SDK	Công bố REST API dưới dạng tool MCP qua stdio hoặc Streamable HTTP, với cơ chế xác thực, giới hạn tốc độ (rate limiting) và phạm vi dự án riêng (Chương 12).
kg-migrate	Go, golang-migrate	Áp dụng 86 migration SQL up/down.
indexer	Python, FastAPI	Cung cấp embedding cho truy vấn và đoạn văn (e5, 384 chiều) và các engine chat dạng streaming của dashboard (fast path và agentic path, Chương 8). Kèm các endpoint vận hành để tạo lại embedding và bổ sung dữ liệu.
worker	Python, asyncio	Tiêu thụ bốn danh sách job Redis mang năm loại công việc: nạp tài liệu và OCR, đồng bộ hệ thống tài liệu (hai loại này dùng chung một danh sách), index mã nguồn, trích xuất và hợp nhất thực thể ở mức chunk, và tạo embedding cho bộ nhớ.

Container	Công nghệ	Trách nhiệm
dashboard	Next.js 16, React 19	Giao diện web và proxy backend-for-frontend (BFF). Trình duyệt không bao giờ gọi trực tiếp Go API (Chương 13).
postgres / redis	PostgreSQL 16, Redis 7	Trạng thái bền vững / danh sách job và trạng thái làm rõ ngắn hạn của agent.



Hình 2. Góc nhìn container. kg-migrate, binary Go thứ ba, là CLI chạy migration schema.

• Bên trong Go API server

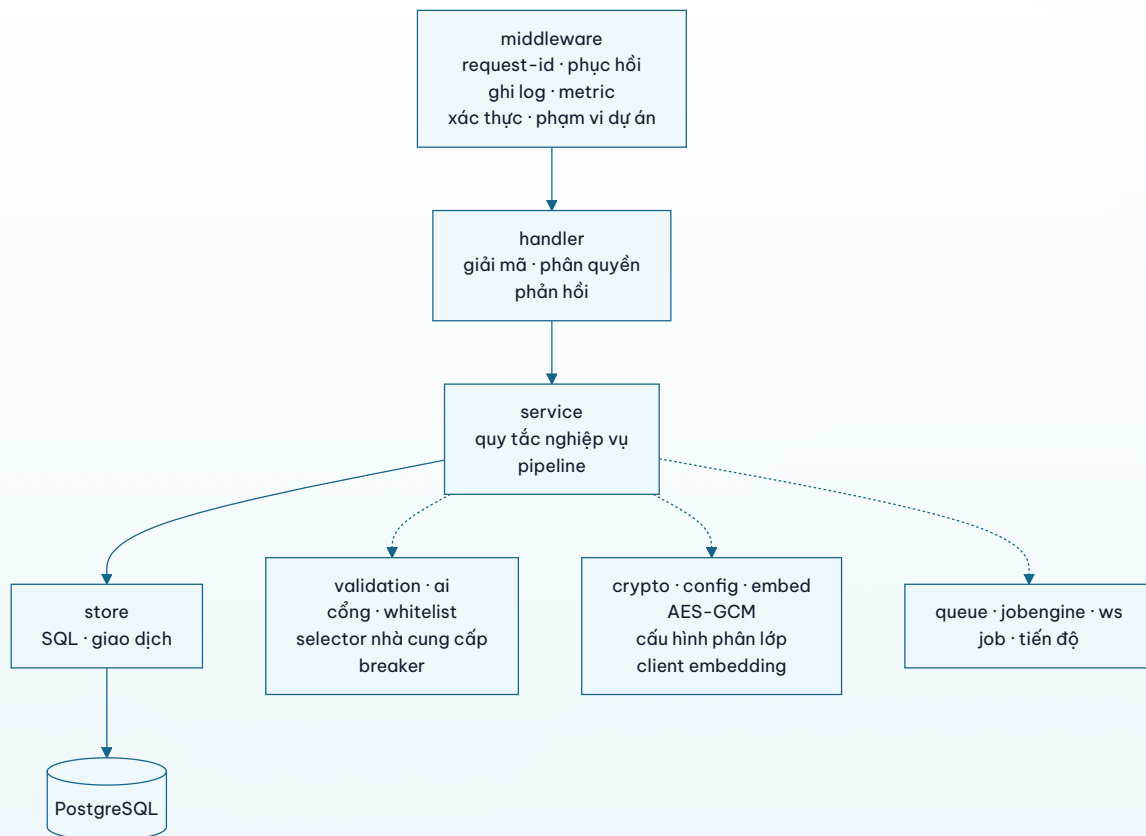
Server theo thiết kế ba tầng kinh điển, được nối bằng constructor injection tại một composition root duy nhất. Handler giải mã yêu cầu và phân quyền; service chứa quy tắc nghiệp vụ, cổng kiểm tra và pipeline; store chứa SQL viết tay và các hàm hỗ trợ giao dịch.

Pipeline xử lý yêu cầu. Mỗi yêu cầu đi qua, từ ngoài vào trong: request-ID → phục hồi khi panic → ghi log truy cập có cấu trúc → metric; các route dưới `/api/` sau đó qua bước xác thực API key và xác định dự án trước khi tới handler. Phạm vi dự án được thực thi bởi middleware đó và trong các handler theo dự án; handler nhận dự án trong thân yêu cầu sẽ kiểm tra lại quyền truy cập theo key của bên gọi, và thao tác ghi bằng key gắn với người dùng yêu cầu vai trò trong dự án (viewer < developer < admin). Áp dụng kiểm tra theo dự án một cách đồng nhất cho mọi thao tác đọc theo ID và cho các key không gắn với người dùng là một hạng mục gia cố trước production (Tài liệu 03, Chương 8). Dự án đã lưu trữ (archived) từ chối mọi method thay đổi dữ liệu.

Tầng nhà cung cấp AI. Nhà cung cấp (Anthropic API, gói thuê bao Claude qua OAuth, endpoint tương thích OpenAI) được đăng ký trong cơ sở dữ liệu với khóa đã mã hóa và một độ ưu tiên. Selector chọn nhà cung cấp đang hoạt động bình thường có độ ưu tiên cao nhất, chuyển dự phòng khi lỗi và bọc mỗi

nhà cung cấp trong một circuit breaker (mở sau 5 lỗi trong 10 phút, nửa mở sau 30 giây). Một bộ phân giải theo chức năng cho phép quản trị viên gán mô hình khác nhau cho các công việc khác nhau. Mỗi lượt gọi đi qua tầng này được ghi log kèm lượng token sử dụng và chi phí để lập ngân sách. Có hai đường gọi nhà cung cấp trực tiếp từ các dịch vụ Python: chat trên dashboard, nhận API key của nhà cung cấp đã chọn từ Go server theo từng yêu cầu, và trích xuất ở mức chunk khi được cấu hình với endpoint riêng; cả hai không đi qua circuit breaker và log sử dụng của tầng này.

Cấu hình. Server đọc cấu hình theo bốn lớp, lớp sau ghi đè lớp trước: `config.yaml` (schema tri thức, whitelist cạnh, quy tắc đầy đủ, cài đặt tìm kiếm, AI, chi phí và hợp nhất), sau đó một tập YAML theo môi trường (development, staging, production, self-hosted), sau đó biến môi trường `KG_*`, cuối cùng là bảng `system_settings` cho các giá trị ghi đè lúc chạy và cờ tính năng.



Hình 3. Phân tầng Go server (22 package nội bộ; handler ≈ 18.6k, service ≈ 21.7k, store ≈ 16.2k dòng, không tính test).

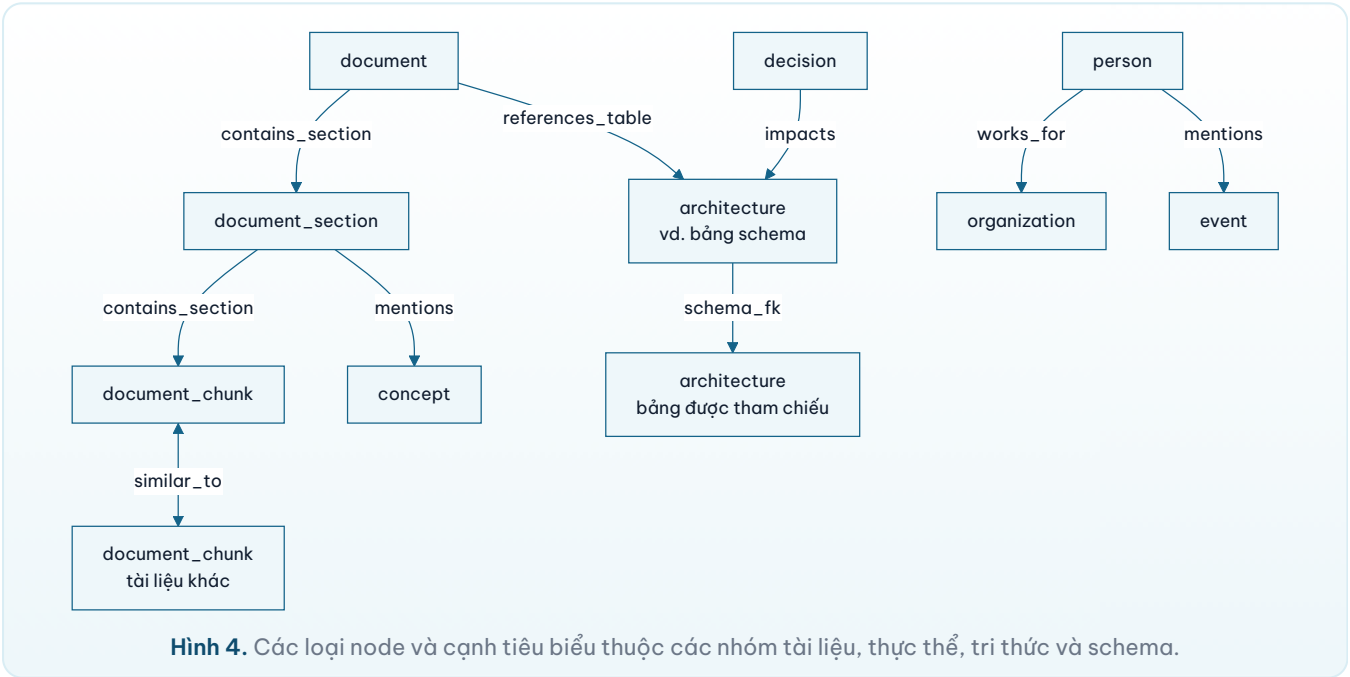
4 Mô hình knowledge graph

Mỗi đơn vị tri thức là một node có kiểu trong một bảng duy nhất, với thuộc tính riêng theo kiểu lưu trong JSONB và schema cho từng kiểu khai báo trong cấu hình. Cạnh có kiểu và phải khớp một whitelist các quy tắc nguồn → quan hệ → đích.

■ Các loại node

Cơ sở dữ liệu thực thi 22 loại node bằng một ràng buộc CHECK; schema thuộc tính, trường bắt buộc và quy tắc đầy đủ của các loại nội dung nằm trong `config.yaml`. Các loại chia thành bốn nhóm:

Nhóm	Loại	Được tạo bởi
Tri thức dự án	decision concept requirement task architecture discovery initiative session	Agent và con người qua MCP / API; sinh graph từ schema CSDL đang vận hành (bảng thành node <code>architecture</code>); index mã nguồn
Tài liệu	document document_section document_chunk dataset external document_ref	Pipeline nạp tài liệu
Thực thể	person organization event location artifact project	Trích xuất bằng LLM ở mức chunk (cùng với <code>concept</code>)
Bản ghi	master_record derived_record	Hệ thống ngoài ghi ngược <code>derived_record</code> (idempotent theo bản ghi nguồn); <code>master_record</code> dành cho thực thể đã hợp nhất

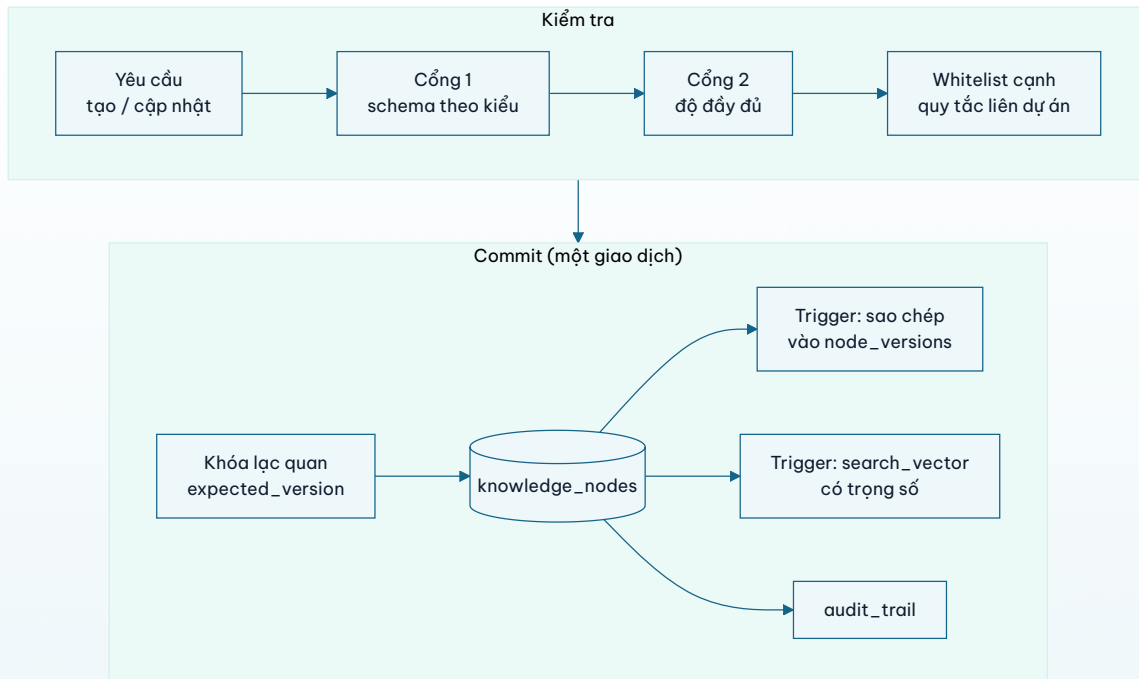


■ **Cạnh và whitelist**

Cạnh có kiểu, thuộc tính JSONB (vd. độ tin cậy và lập luận của quan hệ do AI phát hiện), nguồn gốc (người tạo, phiên) và ràng buộc duy nhất trên (nguồn, đích, kiểu). Kiểu cạnh *cố ý không* bị ràng buộc trong cơ sở dữ liệu: whitelist trong cấu hình là nguồn sự thật duy nhất, gồm **59 quy tắc trên 26 nhãn quan hệ**, mỗi quy tắc có thể kèm quyền liên dự án.

Nhóm	Nhãn
Cấu trúc	contains_section part_of upload_batch
Ngữ nghĩa	mentions related_to relates_to about evidence derived_from similar_to works_for causes
Tri thức kỹ thuật	implements impacts depends_on blocked_by fulfilled_by supersedes references_component
Dữ liệu & liên nguồn	schema_fk schema_implicit schema_many_to_many references_table data_maps_to_table cross_source_reference references_document

■ Cổng ghi và lịch sử



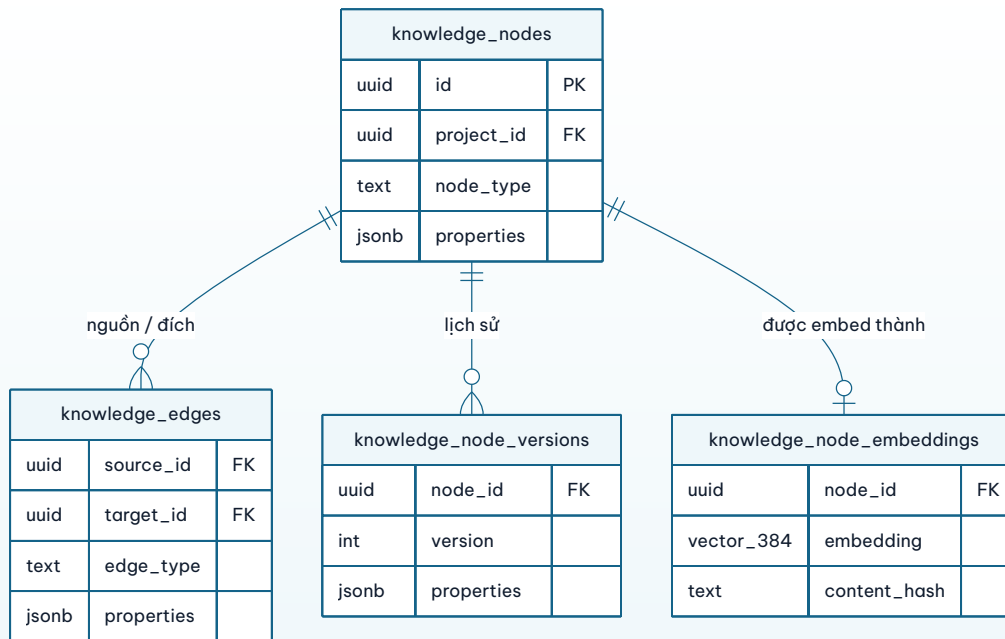
Hình 5. Mọi thao tác ghi graph qua cổng schema, độ đầy đủ, whitelist và khóa lạc quan; lịch sử và chỉ mục tìm kiếm do trigger cơ sở dữ liệu duy trì trong cùng giao dịch.

- **Cổng 1** kiểm tra thuộc tính theo schema của kiểu; **Cổng 2** áp dụng quy tắc và hồ sơ độ đầy đủ (vd. decision phải có lý do).
- **Khóa lạc quan**: cập nhật từng phần có thể kèm `expected_version` ; nếu không khớp, trả HTTP 409 thay vì ghi đè thay đổi đồng thời.
- **Lịch sử phiên bản**: một trigger cập nhật sao chép dòng trước đó vào `knowledge_node_versions` (duy nhất theo node + phiên bản). Node chuyển sang `deprecated` hoặc `superseded` ; API node không có thao tác xóa. Khi một tài liệu đã thay đổi được sinh lại, các mục và chunk cũ của nó, cùng các dòng phiên bản, bị xóa và dựng lại (Chương 6).
- **Vector tìm kiếm**: trigger duy trì `tsvector` có trọng số (tiêu đề và tên cao nhất, rồi trường mô tả, rồi trường ngữ cảnh), cùng chỉ mục trigram trên tiêu đề.

5 Mô hình dữ liệu

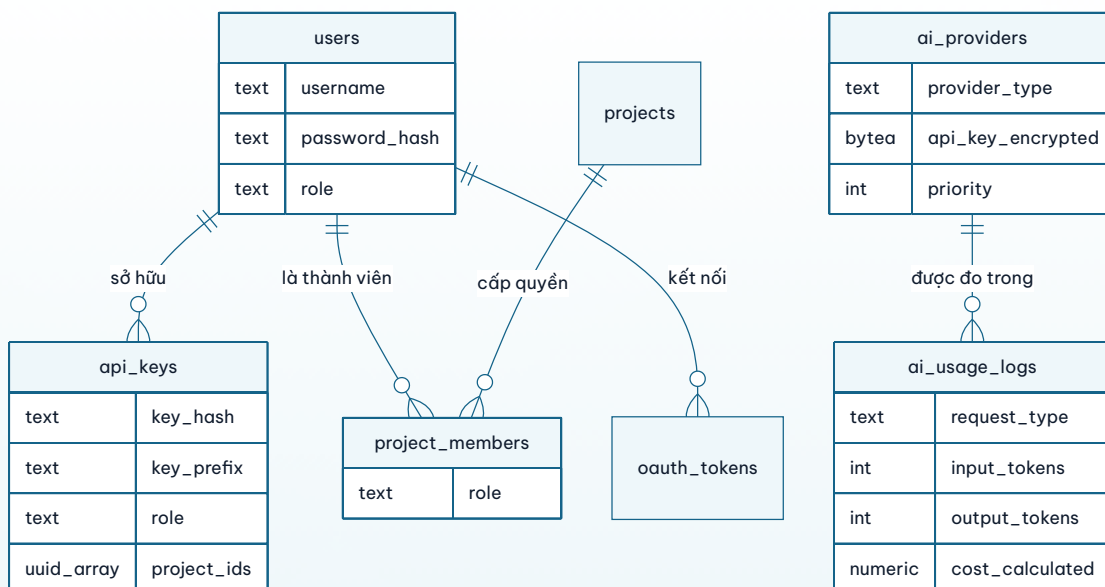
Schema được quản lý bằng 86 cặp migration tiến/lùi và chứa 54 bảng ứng dụng. Ba extension PostgreSQL đảm nhận phần việc nặng: `pgvector` (độ tương đồng vector), `pg_trgm` (so khớp mờ tiêu đề) và `unaccent` (tìm kiếm tiếng Việt không phân biệt dấu). Các sơ đồ dưới đây chia schema theo miền; chỉ hiển thị các bảng và cột chính.

5.1 Lỗi graph



Hình 6. Lỗi graph: một bảng node có kiểu duy nhất với thuộc tính JSONB, cạnh theo whitelist, lịch sử do trigger duy trì và embedding 384 chiều.

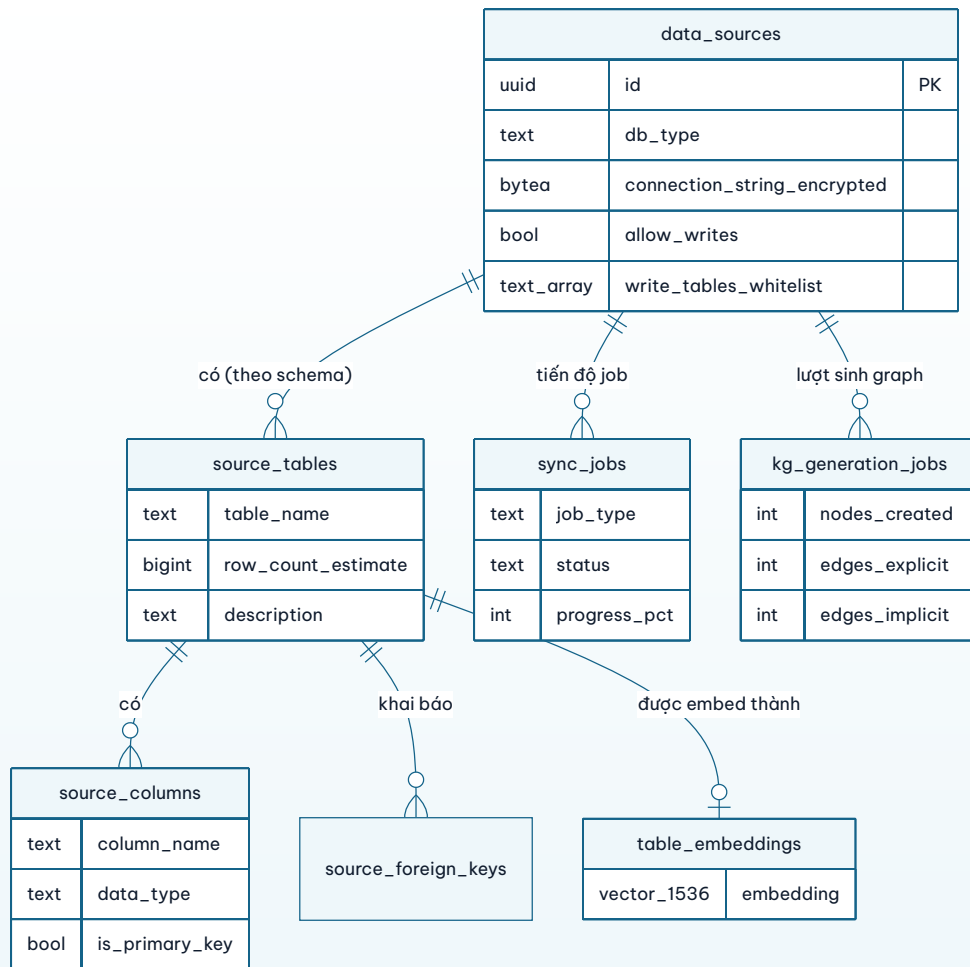
5.2 Định danh, truy cập và quản trị AI



Hình 7. Miền truy cập: API key chỉ được lưu dưới dạng hash; khóa nhà cung cấp và OAuth token được mã hóa; các lượt gọi AI đi qua Go server được đo lường.

5.3 Nguồn dữ liệu đang vận hành và danh mục schema

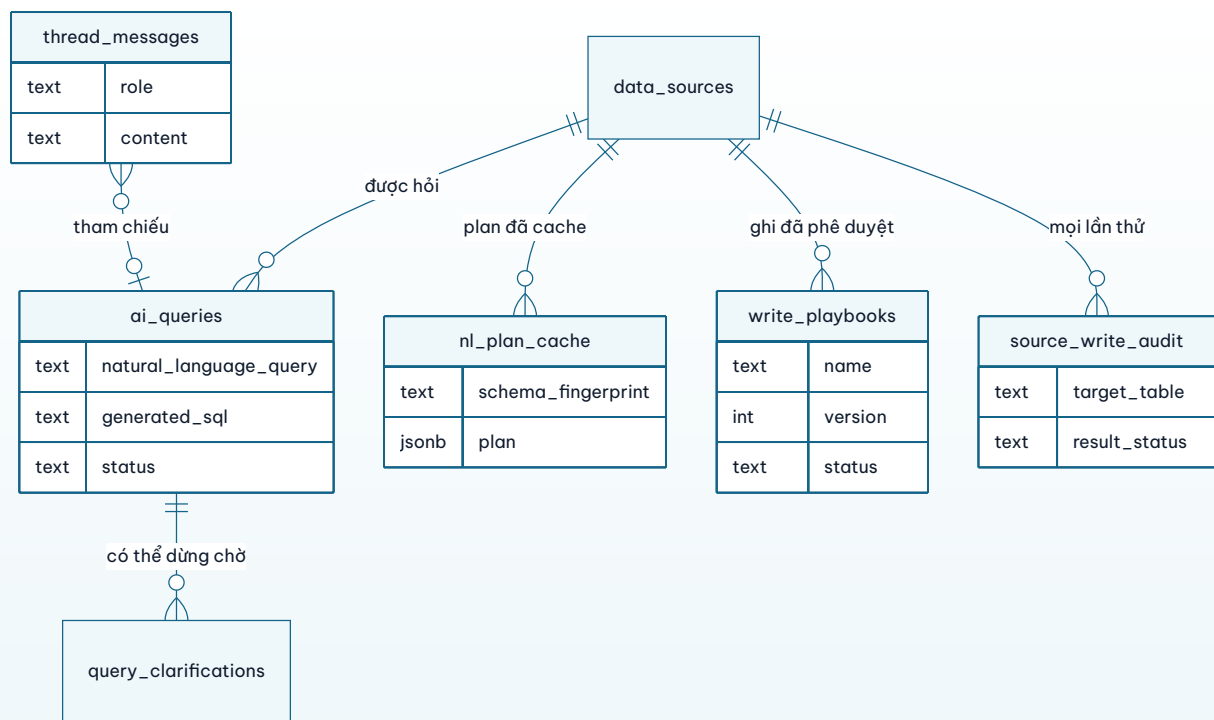
- **Xóa mềm và trạng thái kiểm tra.** Nguồn dữ liệu được xóa mềm và lưu thời điểm cùng kết quả của lần kiểm tra kết nối gần nhất; chuỗi kết nối chỉ được lưu dưới dạng bản mã AES-256-GCM.
- **Ghi đè của con người.** Bảng và cột có cả mô tả do AI sinh và một mô tả riêng của người dùng, nên tri thức đã được biên tập không bao giờ bị ghi đè khi sinh lại.
- **Theo dõi job.** `sync_jobs` theo dõi tiến độ của trích xuất schema, đồng bộ schema, mô tả schema và sinh graph (pending → running → completed / failed / cancelled), với phần trăm tiến độ cấp cho luồng tiến độ trực tiếp của dashboard. Các lượt sinh graph còn ghi số node và số cạnh tường minh, cạnh ngầm và cạnh bị loại vào `kg_generation_jobs`.



Hình 8. Miền nguồn dữ liệu: thông tin kết nối được mã hóa, danh mục schema đã trích xuất và các bảng job điều khiển việc đồng bộ và sinh graph.

5.4 Truy vấn, hội thoại và thao tác ghi

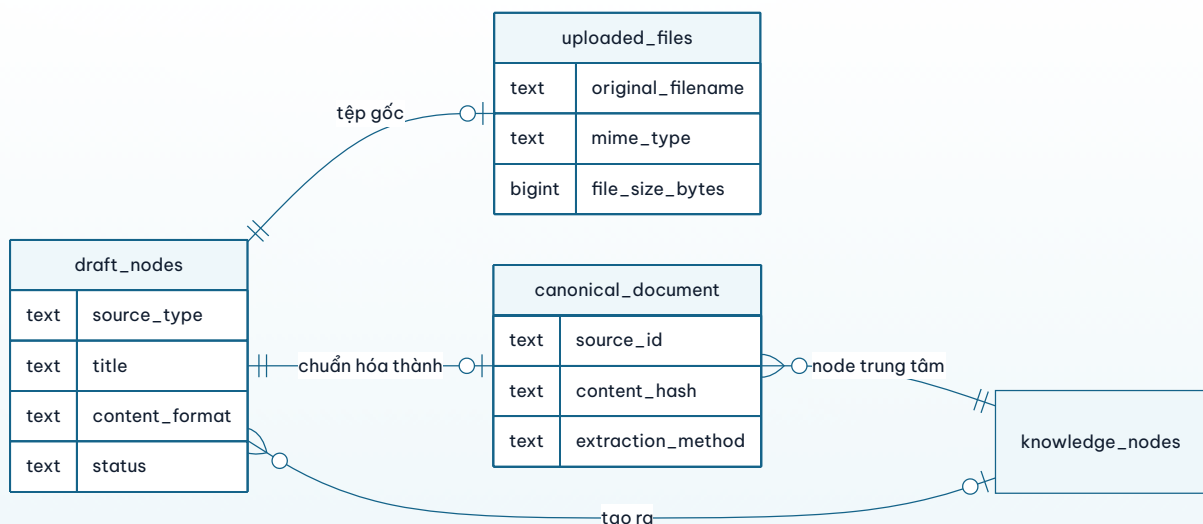
- **Câu trả lời có thể tái lập.** Mỗi truy vấn lưu câu hỏi, SQL đã sinh và kết quả, nên mọi câu trả lời đều truy ngược được tới đúng câu lệnh đã tạo ra nó.
- **Cache query plan theo schema.** Plan được cache theo câu hỏi đã chuẩn hóa và dấu vân tay schema, có đếm số lần trúng; khi schema thay đổi, một dấu vân tay mới được tạo thay vì dùng plan cũ.
- **Playbook là duy nhất theo tên trong mỗi nguồn dữ liệu,** có phiên bản, và ghi lại người phê duyệt, thời điểm phê duyệt, cùng lý do một playbook bị đánh dấu stale.
- **Audit ghi lưu cả đầu vào.** Mỗi lần thử ghi lưu tác nhân, thao tác, bảng đích, giá trị đầu vào và kết quả: thành công, bị một cổng từ chối, hay thất bại tại cơ sở dữ liệu.
- **Hội thoại** lưu số token, độ trễ và mô hình của từng tin nhắn, và liên kết tới truy vấn mà tin nhắn đã chạy; yêu cầu làm rõ trong chat được lưu trong bảng riêng có thời hạn hết hiệu lực.



Hình 9. Miền truy vấn và ghi: mọi truy vấn ngôn ngữ tự nhiên, plan đã cache, playbook và lần thử ghi đều được lưu lại.

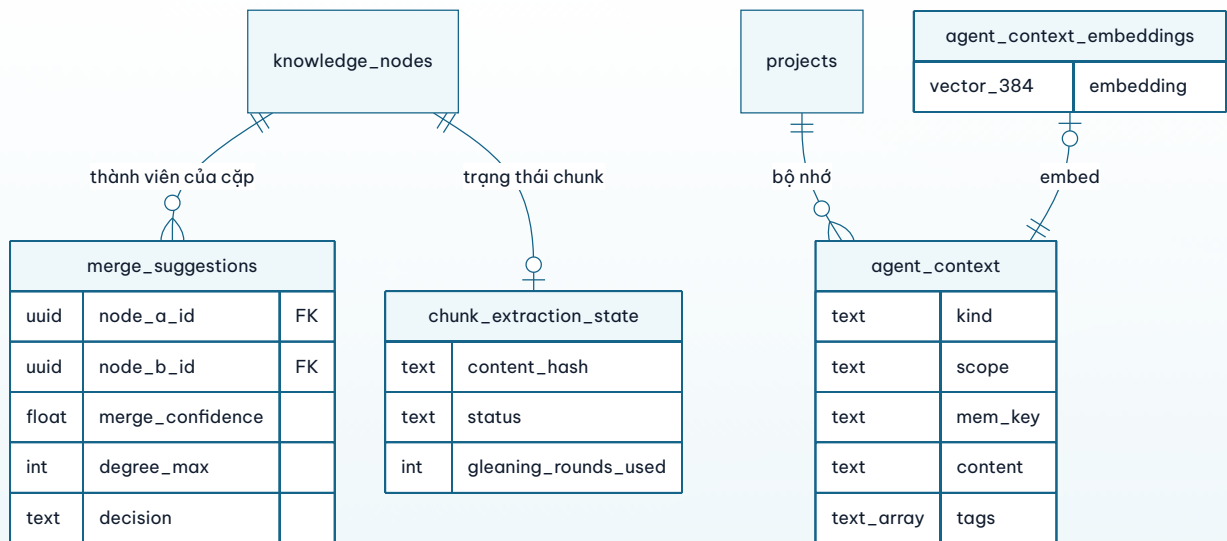
5.5 Tài liệu và nạp dữ liệu

- **Mỗi mục nguồn một bản nháp.** Bản nháp là duy nhất theo (dự án, loại nguồn, ID nguồn), nên gửi lại cùng một mục sẽ cập nhật thay vì tạo bản trùng; mỗi bản nháp liên kết tới node graph mà nó tạo ra và ghi lại người đã phê duyệt.
- **Mỗi bản nháp một bản ghi chuẩn hóa,** mang content hash SHA-256 được dùng bởi ba tầng khử trùng lặp. Bản ghi chuẩn hóa và tệp đã tải lên được xóa mềm, nhờ đó một tài liệu có thể quay về phiên bản trước một cách gọn gàng.
- **Kết nối nguồn** (không vẽ trong hình) lưu cấu hình connector và trạng thái theo từng dự án, mọi thông tin xác thực đều được mã hóa.



Hình 10. Miền nạp dữ liệu: mọi đầu vào đều trở thành một bản nháp; tài liệu chuẩn hóa mang content hash dùng để khử trùng lặp theo nguyên tắc "chỉ nạp một lần".

5.6 Hợp nhất thực thể và bộ nhớ agent



Hình 11. Hợp nhất và bộ nhớ: ứng viên gộp được ghi lại trước khi bất kỳ node nào bị thay đổi; bộ nhớ được đánh chỉ mục lại.

6 Pipeline nạp dữ liệu

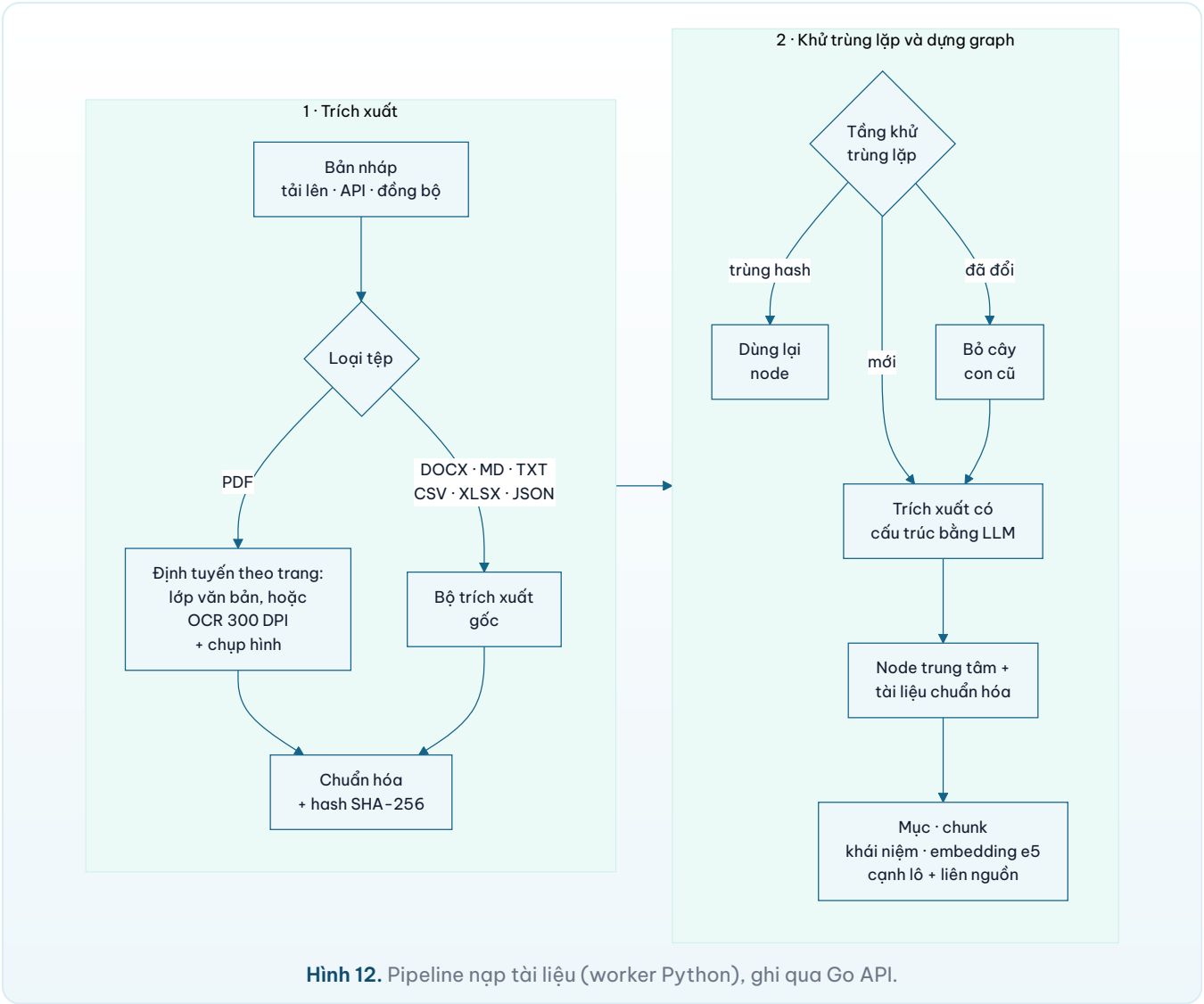
Tri thức vào graph theo ba đường: tài liệu (tệp và bản nháp có cấu trúc), cơ sở dữ liệu đang vận hành (danh mục schema) và mã nguồn (symbol). Cả ba đều ghi qua Go API, nên mọi node đều qua cùng các cổng kiểm tra.

6.1 Tài liệu LIVE

Điểm vào. Kéo-thả tải lên trên dashboard; một MCP client tải tệp lên endpoint tệp của bridge theo kênh riêng (out-of-band); API nạp công khai (một bản nháp hoặc lô tới 50); và đồng bộ với hệ thống quản lý tài liệu, được cấu hình như một kết nối nguồn của dự án, lần lượt duyệt qua các tài liệu đã phân tích, tải từng tài liệu qua một URL có chữ ký, nạp vào rồi xóa bản sao tạm. Mọi đầu vào đều trở thành một **node nháp**. Bản nháp được xử lý ngay hoặc giữ lại chờ phê duyệt tùy theo cài đặt dự án.

Định dạng	Trích xuất	Kết quả trong graph
PDF	Lớp văn bản pypdf theo từng trang; trang có < 50 ký tự, hoặc có hình ảnh và văn bản tiếng Việt bị lỗi, được render ở 300 DPI và OCR bằng Tesseract (vie). Tối đa 50 trang mỗi tệp.	Markdown → mục, chunk, khái niệm, embedding
DOCX	Đoạn văn python-docx	Mục, chunk, khái niệm, embedding
Markdown, TXT	Đọc trực tiếp	Mục, chunk, khái niệm, embedding
CSV, XLSX, JSON	Chuyển thành văn bản JSON / CSV; cắt bỏ các dòng trống ở cuối. Nhận diện tất định bảng cân đối kế toán và báo cáo kết quả kinh doanh theo mẫu Thông tư 200 của Việt Nam với tiêu đề tiếng Anh.	Node trung tâm (<code>dataset</code> hoặc <code>document</code>) kèm tóm tắt và thuộc tính có cấu trúc

Kênh phụ OCR. Trên các trang scan, RapidOCR (ONNX) trích xuất số hiệu văn bản, ngày, diện tích, số tiền và số định danh bằng các quy tắc mẫu, với Tesseract làm phương án dự phòng cho trường hợp còn thiếu. Các số liệu thu được được lưu thành trường có cấu trúc và nối vào văn bản của tài liệu, nhờ đó có thể tìm kiếm được. Không dùng mô hình thị giác.



Hình 12. Pipeline nạp tài liệu (worker Python), ghi qua Go API.

Khử trùng lặp ("chỉ nạp một lần"). Nội dung được chuẩn hóa (BOM, ký tự xuống dòng, khoảng trắng) rồi băm. Có ba tầng: (1) cùng nguồn, cùng hash thì dừng lại node; (2) cùng nguồn, hash thay đổi thì sinh lại: các mục, chunk và embedding cũ bị xóa và dựng lại trên cùng node trung tâm; (3) cùng hash từ bất kỳ nguồn nào khác thì dừng lại node đó. Nếu tra cứu gặp lỗi, bản nháp bị đánh dấu thất bại thay vì chấp nhận rủi ro tạo bản trùng một cách âm thầm.

Cấu trúc và embedding. Cây tiêu đề Markdown tạo ra tối đa 200 node `document_section` mỗi tài liệu. Một mục dài tới khoảng 1,500 ký tự trở thành một `document_chunk` duy nhất; mục dài hơn được gom theo đoạn văn thành các chunk tối đa 1,200 ký tự, tách theo câu khi cần. Mỗi chunk mang offset dòng và ký tự, nên câu trả lời có thể trích dẫn đúng đoạn văn. Thực thể từ bước trích xuất trở thành node `concept` trong phạm vi dự án, được khử trùng lặp theo một khóa đã bỏ dấu, đồng thời chuẩn hóa các chữ viết tắt loại hình pháp lý của Việt Nam. Việc dừng lại khái niệm giữa các tài liệu chính là thứ kết nối các tài liệu với nhau. Mục và chunk được embed bằng multilingual-e5-small (384 chiều, chuẩn hóa L2, tiền tố `passage:`) theo lô 32.

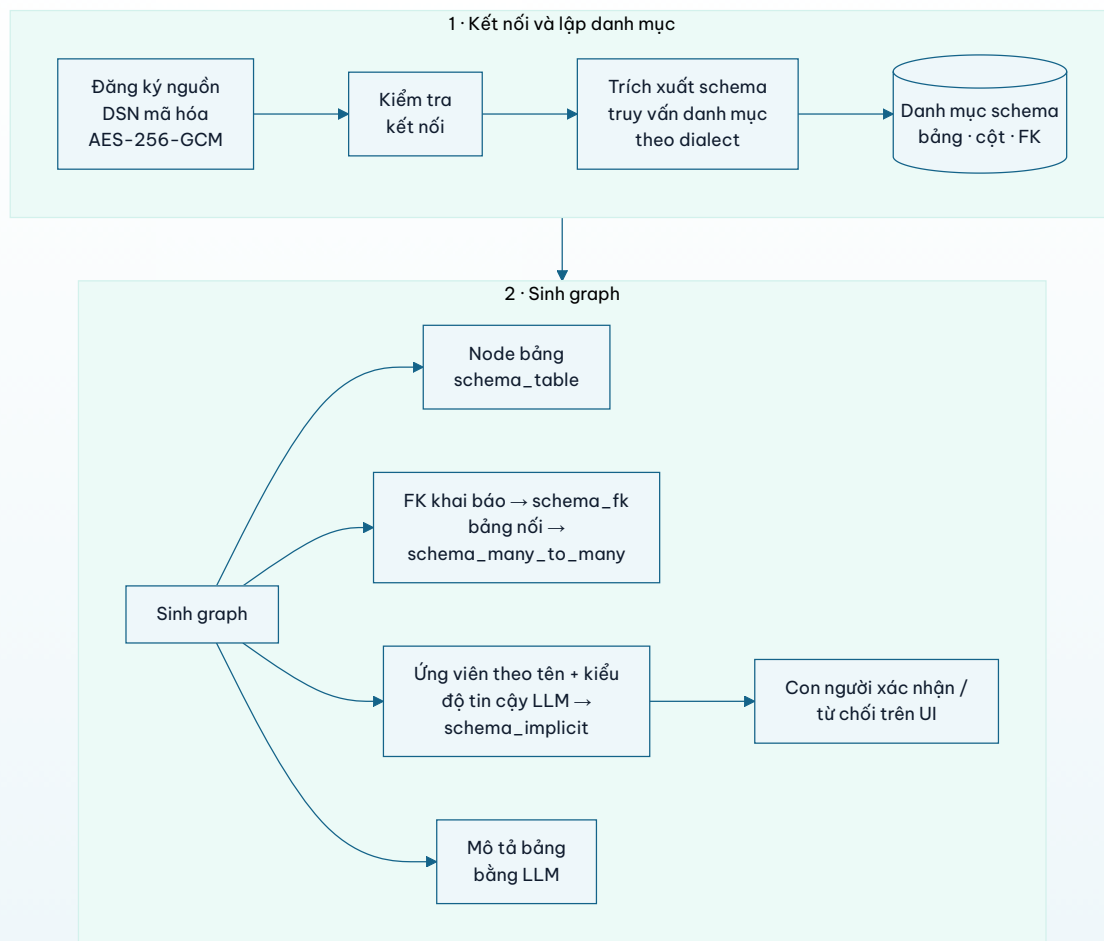
Trích xuất có cấu trúc. Một lượt gọi LLM cho mỗi tài liệu (temperature 0, định tuyến qua tầng AI của Go) trả về chủ đề, thực thể, quyết định, bảng được tham chiếu, API được tham chiếu và những người

được nhắc tới, giữ nguyên tên theo ngôn ngữ gốc. Lượt này đọc 12,000 ký tự đầu của tài liệu. Một lượt LLM thứ hai đề xuất cạnh liên nguồn (`cross_source_reference` , `references_document` , `references_component`) với tối đa 80 node architecture, và áp dụng những cạnh có độ tin cậy ≥ 0.5 . Các bảng được nêu trong tài liệu được liên kết tới node schema của chúng bằng `references_table` .

6.2 Cơ sở dữ liệu đang vận hành LIVE

Kết nối một cơ sở dữ liệu sẽ biến schema của nó thành một phần có thể điều hướng của graph. Cả trích xuất và sinh graph đều chạy bên trong Go server: trích xuất trên job engine trong tiến trình (3 worker, 3 lần thử lại), sinh graph như một job nền, với tiến độ được stream trực tiếp tới dashboard.

- **Loại CSDL:** PostgreSQL (`lib/pq`), MySQL và MariaDB (`go-sql-driver/mysql`), Microsoft SQL Server (`go-mssqldb`), mỗi loại có bộ truy vấn danh mục riêng.
- **Quan hệ tương minh** lấy từ khóa ngoại đã khai báo; bảng nối được phát hiện và chuyển thành cạnh nhiều-nhiều.
- **Quan hệ ngầm** được đề xuất dựa trên quy ước đặt tên và kiểu cột tương thích, được LLM chấm điểm 0–1 và lưu kèm độ tin cậy và lập luận. Con người có thể xác nhận, từ chối hoặc đảo ngược một lần từ chối trên UI.
- **Đồng bộ lại** làm mới danh mục, và mỗi lần đồng bộ thành công sẽ kiểm tra lại các playbook ghi đã phê duyệt theo schema mới (Chương 9).
- Các node `schema_table` tạo ra cũng là thứ các engine truy vấn dùng để thu hẹp câu hỏi về các bảng liên quan (Chương 8), và là thứ các tài liệu liên kết tới.



Hình 13. Từ một cơ sở dữ liệu đang vận hành tới knowledge graph của schema.

6.3 Mã nguồn BUILT

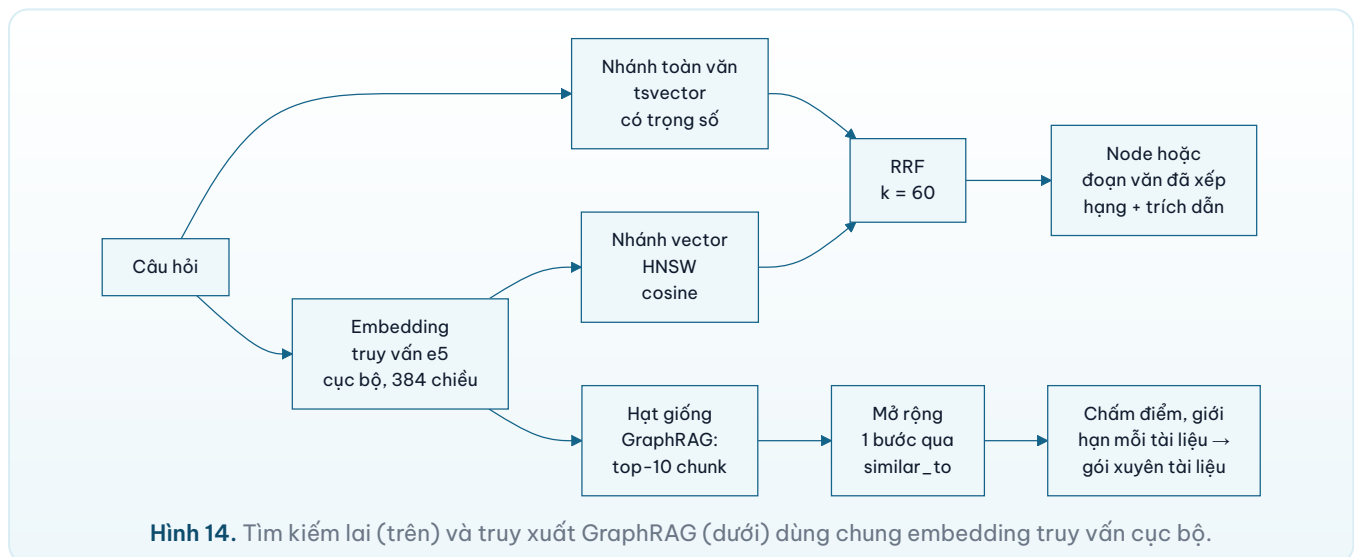
Một code indexer (package và CLI Python, cũng chạy được qua MCP bridge như một tool cục bộ trên máy của agent) phân tích repository bằng tree-sitter và lưu mỗi symbol thành một node `architecture` mang đường dẫn tệp, khoảng dòng, chữ ký và hash thân hàm.

Ngôn ngữ	Symbol được trích xuất
TypeScript / JavaScript	hàm, lớp, method, interface, type alias, export, component React, route Next.js
Python	hàm, method, lớp, route FastAPI, decorator, docstring
Go	hàm, method (kèm receiver), struct, interface, type alias, hằng, biến
Dart	lớp, mixin, enum, extension, typedef, hàm, thành viên
Markdown	node trung tâm tài liệu và các mục theo tiêu đề

Index lại theo kiểu tăng dần: symbol được định khóa theo tệp, tên và loại, còn hash thân hàm phát hiện thay đổi, nên chỉ symbol mới, đã đổi hoặc đã xóa mới chạm tới graph. Bộ quét tuân theo `.gitignore`. Hiện indexer ghi nhận quan hệ chứa (lớp → method, tài liệu → mục); cạnh import và call graph chưa được tạo.

7 Truy xuất: tìm kiếm, xếp hạng lại & GraphRAG

Truy xuất chạy trong PostgreSQL: tìm kiếm từ vựng và tìm kiếm vector dùng cùng các bảng và cùng ranh giới giao dịch với graph, và một bước trộn kết quả kết hợp hai nhánh. Cấu trúc graph sau đó mở rộng kết quả sang các đoạn văn liên quan trong tài liệu khác.



Chế độ tìm kiếm

Chế độ	Cơ chế
Toàn văn (mặc định)	<code>tsvector</code> có trọng số do trigger duy trì (tiêu đề/tên có trọng số cao nhất); các từ trong truy vấn kết hợp theo ngữ nghĩa OR; <code>ts_rank</code> chuẩn hóa về 0–1. Lịch sử hội thoại có chỉ mục riêng không phân biệt dấu (<code>unaccent</code>), phù hợp với tiếng Việt.
Ngữ nghĩa	Câu hỏi được mô hình e5 cục bộ embed (tiền tố <code>query:</code>) và so khớp theo khoảng cách cosine trên chỉ mục HNSW của embedding node; phạm vi mặc định là mục, tài liệu và khái niệm.

Chế độ	Cơ chế
Lai	Nhánh toàn văn và nhánh ngữ nghĩa chạy đồng thời và được trộn kết quả bằng Reciprocal Rank Fusion ($k = 60$), khử trùng lặp theo node và phân định hòa theo độ mới. Hệ thống suy giảm êm: nếu embedding lỗi thì chỉ trả kết quả toàn văn; nếu một nhánh lỗi thì nhánh kia vẫn trả lời.
Tìm đoạn văn	Tìm kiếm lai giới hạn trong <code>document_chunk</code> ; trả về nguyên văn đoạn văn kèm mục và khoảng dòng để agent có thể trích dẫn.
Truy vấn có cấu trúc	Bộ lọc JSON theo loại node, trạng thái và thuộc tính, tùy chọn mở rộng nhiều bước.

Điều hướng graph

- Láng giềng:** một bước theo cả hai chiều, có phân trang. **Duyệt:** common table expression đệ quy trong SQL, tối đa ba bước, lọc được theo loại cạnh.
- Điều hướng tài liệu:** metadata tài liệu để trích dẫn, dàn ý các mục, nội dung mục, mục cha/con/anh em, và liên kết ngược từ các nguồn khác.
- Tài liệu liên quan:** tài liệu được xếp hạng theo các thực thể chuẩn mà chúng có chung, thực thể càng hiếm càng được đánh trọng số cao; kèm danh sách thực thể mà hai tài liệu có chung.

GraphRAG

Truy xuất xuyên tài liệu dựa trên các cạnh `similar_to` giữa các chunk thuộc những tài liệu *khác nhau*. Một worker liên kết chạy nền mỗi năm phút trên các dự án vừa nạp dữ liệu (hoặc theo yêu cầu), tìm các láng giềng xuyên tài liệu gần nhất của mỗi chunk, giữ tối đa 5 láng giềng cho mỗi chunk có độ tương đồng cosine trên 0.90 và ghi chúng thành cạnh. Khi truy vấn, bộ truy xuất gieo hạt bằng 10 chunk liên quan nhất (dùng `ef_search` của HNSW nâng lên 400 để tập chunk không bị thiếu hụt do lọc sau), mở rộng một bước qua `similar_to` , chấm điểm mỗi láng giềng bằng độ liên quan của hạt giống $\times$ độ tương đồng cạnh, giới hạn số kết quả mỗi tài liệu, và có thể làm giàu gói kết quả bằng mục cha, mở rộng theo thực thể chung và đoạn trích nội tuyến.

Chỉ mục vector

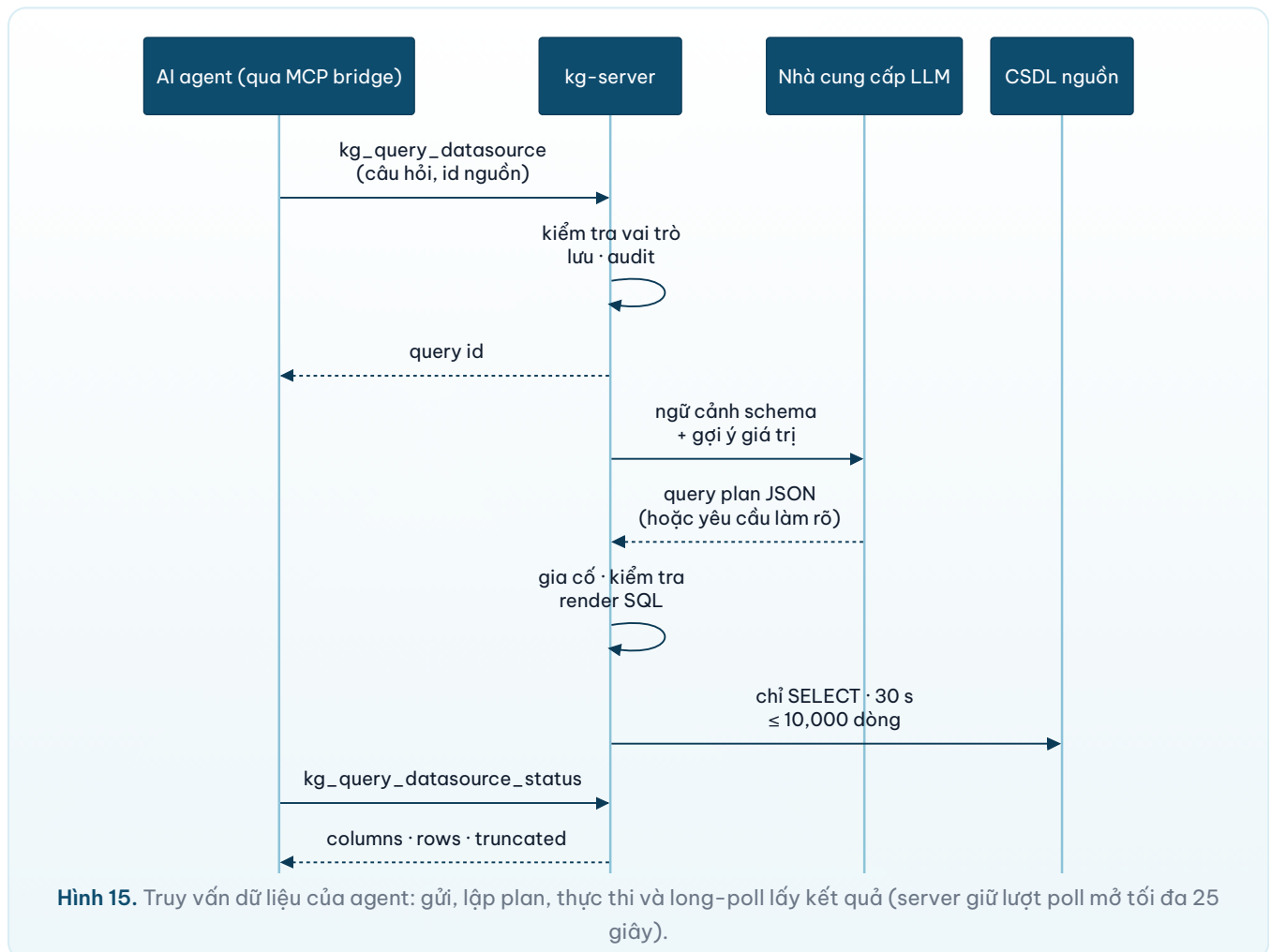
Cột	Số chiều	Chỉ mục	Mô hình	Dùng cho
<code>knowledge_node_embeddings</code>	384	HNSW, cosine	multilingual-e5-small (cục bộ)	Tìm kiếm ngữ nghĩa & lai, đoạn văn, GraphRAG, phân khối thực thể
<code>agent_context_embeddings</code>	384	HNSW, cosine	multilingual-e5-small (cục bộ)	Truy hồi bộ nhớ agent
<code>table_embeddings</code>	1,536	IVFFlat, cosine	API embedding bên ngoài (tùy chọn)	Chọn bảng cho truy vấn dữ liệu trên schema lớn BUILT

8 Truy vấn dữ liệu bằng ngôn ngữ tự nhiên

Agent đặt câu hỏi với cơ sở dữ liệu đang vận hành bằng ngôn ngữ thông thường. Trên đường agent / API này, LLM không bao giờ viết câu SQL được thực thi: nó tạo ra một **query plan** có cấu trúc, rồi server gia cố, kiểm tra plan đó theo schema đã trích xuất và render thành SQL tham số hóa, chỉ đọc. Các chế độ chat của dashboard được mô tả ở cuối chương.

■ Các giai đoạn pipeline

- 1. Cache query plan.** Plan được cache theo câu hỏi đã chuẩn hóa cộng dấu vân tay của schema và prompt, nên câu hỏi lặp lại bỏ qua LLM và thay đổi schema làm vô hiệu các plan cũ.
- 2. Ngữ cảnh.** Bộ lập plan dựng ngữ cảnh schema từ danh mục đã trích xuất, bổ sung *gợi ý giá trị* phân loại được lấy mẫu chỉ đọc từ nguồn để bộ lọc dùng đúng giá trị thực. Khi embedding bảng được cấu hình, các bảng liên quan nhất được chọn theo độ tương đồng vector.
- 3. Lập plan.** LLM (do selector nhà cung cấp chọn, có failover) trả về một plan JSON (bảng, phép nối, bộ lọc, phép tổng hợp, nhóm, having, sắp xếp, giới hạn) hoặc yêu cầu làm rõ.
- 4. Gia cố và sửa lỗi.** Các bước tắt định sửa alias, chuyển bảng lác thành phép nối, khử trùng lặp và kiểm tra điều kiện nối, đối chiếu mọi cột với schema đã trích xuất, giới hạn truy vấn xếp hạng ở 100 dòng và sửa chiều xếp hạng. Một lần thực thi thất bại được thử lại một lần với dạng đơn giản hóa.
- 5. Render.** Bộ sinh SQL xuất SQL theo dialect với mọi giá trị được bind làm tham số; toán tử lấy từ allow-list, và các dạng cột, khoảng thời gian và truy vấn con chỉ được chấp nhận qua các ngữ pháp đóng.
- 6. Thực thi và lưu.** Kết quả, trạng thái và SQL đã sinh được lưu cùng bản ghi truy vấn để bên gọi lấy về và để audit.



Guard khi thực thi (đường agent)	Thiết lập
Dạng câu lệnh	Phải bắt đầu bằng <code>SELECT</code> hoặc <code>WITH</code>
Chế độ phiên	Nguồn PostgreSQL: phiên được đặt <code>READ ONLY</code>

Guard khi thực thi (đường agent)	Thiết lập
Kết nối	Pool một kết nối mới cho mỗi truy vấn; thông tin xác thực được giải mã khi dùng
Thời gian chờ	30 giây mỗi truy vấn
Giới hạn kết quả	10,000 dòng, kèm cờ <code>truncated</code> ; bên gọi có thể giới hạn số dòng nhận về bằng <code>max_rows</code>
Giá trị	Luôn là tham số được bind, không bao giờ nối chuỗi
Lưu vết	Mọi lần gửi được lưu trong <code>ai_queries</code> và ghi vào nhật ký audit

Làm rõ thay vì đoán

Khi câu hỏi mơ hồ (ví dụ có nhiều chỉ số hoặc cột ngày có thể phù hợp), bộ lập plan trả về `clarification_needed` kèm câu hỏi cần đặt cho người dùng, thay vì chạy một truy vấn trông hợp lý nhưng sai. Trạng thái này được lưu lại, nên agent có thể hỏi người dùng rồi gửi lại.

Phân tích dạng hội thoại trên dashboard

Chat của dashboard dùng hai engine streaming trong Python indexer. Go server làm proxy cho luồng Server-Sent Events (tối đa 3 luồng chat đồng thời cho mỗi người dùng), chuyển kèm ngữ cảnh nhà cung cấp và nguồn dữ liệu theo từng yêu cầu: API key của nhà cung cấp và chuỗi kết nối đã mã hóa. Indexer giải mã chuỗi kết nối và kết nối trực tiếp tới cơ sở dữ liệu nguồn. Các engine này dựa vào bộ kiểm tra SQL và giới hạn số dòng của riêng chúng; chúng không đặt phiên chỉ đọc ở mức cơ sở dữ liệu (bổ sung phiên này là một hạng mục gia cố, Tài liệu 03, Chương 8).

Engine	Cách hoạt động
Fast path	Thu hẹp schema bằng các node <code>schema_table</code> của graph và từ khóa trong câu hỏi; LLM điền vào một tool query plan có cấu trúc; SQL được render tắt định với giá trị lọc được bind và giới hạn số dòng; sau đó kết quả được tóm tắt, một định dạng trình bày được chọn (văn bản, bảng hoặc biểu đồ), và nhận định cùng câu hỏi tiếp theo gợi ý được stream dưới dạng các khối có kiểu.
Agentic path	Một vòng lặp dùng tool với hai mức <i>quick</i> (3 vòng, 30k token) và <i>deep</i> (12 vòng, 100k token). Tool: tìm kiếm graph, schema bảng, thực thi SQL, vẽ biểu đồ và làm rõ; mức <i>deep</i> bổ sung lắng nghe, duyệt graph, cấu trúc tài liệu và tìm kiếm ngữ nghĩa. Ở đây mô hình tự viết câu lệnh SELECT; câu lệnh phải qua một bộ kiểm tra chỉ đọc (chỉ câu lệnh đọc, một câu lệnh, chặn từ khóa ghi/DDDL, giới hạn độ dài), và kết quả được cắt còn 50 dòng; một guard chặn vòng lặp ngăn các lượt gọi lặp lại. Một yêu cầu làm rõ sẽ tạm dừng agent, lưu trạng thái trong 10 phút và tiếp tục đúng một lần.

9 Ghi có kiểm soát LIVE

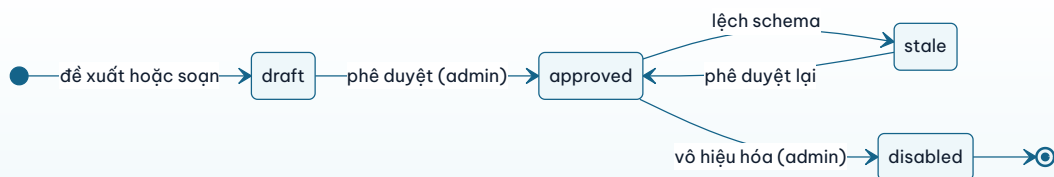
Ghi vào cơ sở dữ liệu của khách hàng mặc định bị tắt, và khi được bật thì chỉ chạy qua hai đường hẹp: chèn một dòng đã kiểm tra hợp lệ, và các **playbook** nhiều bước có tên, phải được phê duyệt trước khi có thể thực thi. Mọi lần thử đều được audit. Đường ghi này đến nay mới được chạy ở quy mô nhỏ.

■ Chèn một dòng

1. **Bật tường minh theo từng nguồn dữ liệu:** `allow_writes` mặc định là false và do quản trị viên thiết lập.
2. **Whitelist bảng tường minh:** whitelist rỗng nghĩa là không bảng nào được ghi.
3. **Kiểm tra theo schema đã trích xuất:** đủ các cột bắt buộc, không có cột lạ, kiểm tra kiểu và độ dài.
4. **Kết nối ghi riêng** với thời gian chờ 30 giây, chỉ dùng một lần.
5. **Audit mọi lần thử:** các lần thử thành công, bị từ chối và thất bại đều được ghi vào `source_write_audit`.

Agent tìm hiểu những gì mình được phép ghi bằng `kg_describe_table` (đặc tả cột) trước khi gọi `kg_insert_datasource_row`.

■ Playbook ghi



Hình 16. Vòng đời playbook. Chỉ playbook đã phê duyệt mới được thực thi (vai trò developer; mọi bước trong một giao dịch); lịch schema tự động đưa playbook ra khỏi hoạt động.

- Playbook là một định nghĩa JSON có tên và phiên bản, mô tả đầu vào và các bước, dùng một bộ từ vựng template nhỏ (`$input.*` , `$item.*` với `foreach` , ID được sinh, `$now`). Ví dụ, nó có thể biểu diễn một dòng tiêu đề cùng các dòng chi tiết của nó.
- **Đề xuất là tất định:** server suy ra một bản nháp từ quan hệ bảng tiêu đề/chi tiết trong schema, không dùng LLM.
- **Thực thi là nguyên tử:** mọi bước chạy trong một giao dịch cơ sở dữ liệu và cùng được hoàn tác khi có bất kỳ lỗi nào.
- **Bảo vệ khỏi lịch schema:** sau mỗi lần đồng bộ schema thành công, playbook được kiểm tra lại và những playbook không còn khớp bị đánh dấu `stale`.
- **Vai trò:** với key gắn với người dùng, phê duyệt, vô hiệu hóa và xóa yêu cầu admin của dự án, còn thực thi yêu cầu developer. Mở rộng các kiểm tra vai trò này cho các key không gắn với người dùng là một hạng mục gia cố trước production (Tài liệu 03, Chương 8).

Các đường ghi hiện hỗ trợ nguồn PostgreSQL. Cả hai tool ghi MCP đều không mang gợi ý chỉ đọc, nên client có thể yêu cầu người dùng xác nhận, và token bridge chỉ đọc không thể gọi chúng.

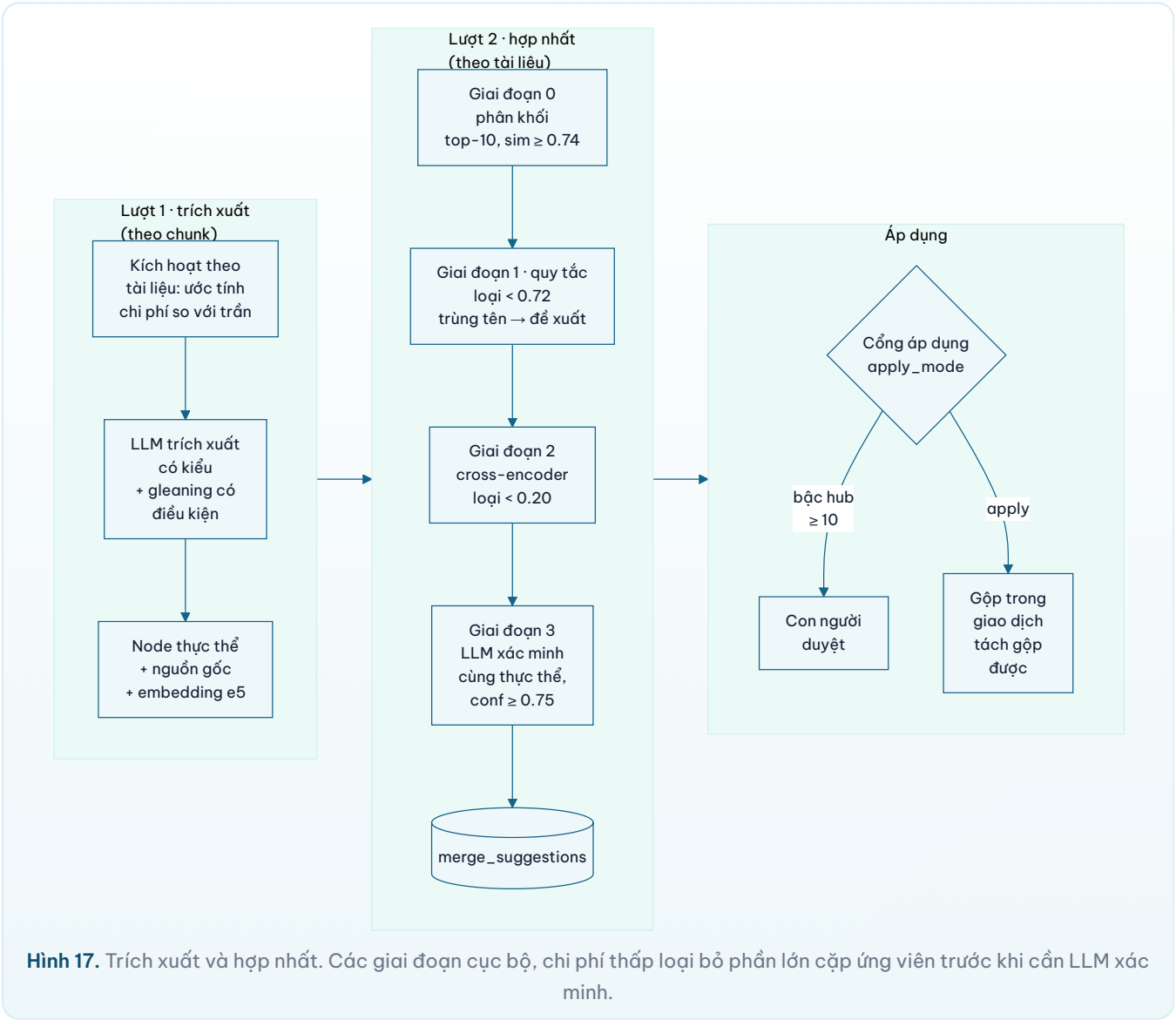
Ghi vào graph tuân theo cùng kỷ luật duyệt: agent gửi tri thức dưới dạng **bản nháp** (`kg_ingest_node` , hoặc `kg_ingest_batch` cho tối đa 50), được phê duyệt từng cái hoặc hàng loạt rồi mới được xử lý thành node.

10 Hợp nhất thực thể & bản ghi chủ LIVE

Tài liệu nhắc tới cùng một người, công ty hay địa điểm với nhiều cách viết. Một pipeline hai lượt trích xuất thực thể có kiểu theo từng chunk, sau đó quyết định, qua các giai đoạn tốn kém dần, thực thể nào là một, và ghi lại mọi quyết định trước khi gộp bất cứ thứ gì.

•
 Lượt 1: trích xuất

Trích xuất được khởi động theo từng tài liệu qua API. Trước tiên server ước tính chi phí (số chunk × số token trung bình × đơn giá mô hình × số vòng gleaning) và từ chối các lượt chạy vượt trần đã cấu hình (\$5 mỗi lượt chạy, \$1 mỗi tài liệu); các chunk đã được trích xuất với cùng content hash sẽ được bỏ qua. Mỗi chunk sau đó được đưa vào hàng đợi như một job riêng. LLM trả về thực thể thuộc tám loại (người, tổ chức, khái niệm, sự kiện, địa điểm, artifact, tham chiếu tài liệu, dự án) và quan hệ từ một bộ từ vựng đóng (works_for , part_of , mentions , causes , related_to , derived_from , evidence), giữ nguyên tên theo ngôn ngữ gốc. Một lượt *gleaning* (tối đa 2 vòng) chỉ chạy trên các chunk dài và thưa thực thể, và dừng ngay khi một vòng không tìm thấy gì mới. Mỗi thực thể giữ nguồn gốc của nó: tài liệu nguồn, chunk và khoảng câu.



•
 Lượt 2: các giai đoạn hợp nhất

Giai đoạn	Quy tắc	Chi phí
0 · Phân khối	Embed mỗi thực thể (tên + mô tả); lấy 10 láng giềng gần nhất có độ tương đồng cosine trên 0.74.	Cục bộ

Giai đoạn	Quy tắc	Chi phí
1 · Quy tắc	Loại dưới 0.72. Đề xuất gộp (độ tin cậy 0.99) khi trùng khớp chính xác sau chuẩn hóa (Unicode NFC, chữ hoa/thường, dấu phân cách, bỏ danh xưng). Không có quy tắc gộp nào khác.	Cục bộ
2 · Cross-encoder	<code>bge-reranker-v2-m3</code> chạy cục bộ chấm điểm cặp; dưới 0.20 bị loại. Nó không bao giờ tự mình chấp thuận một lần gộp.	Cục bộ, không tốn phí API
3 · LLM xác minh	Các cặp được gom lô sáu cặp một lần (đồng thời 10); chỉ ghi đề xuất khi mô hình đánh giá là cùng một thực thể với độ tin cậy ≥ 0.75 .	LLM
Tùy chọn · thẩm định	Khi được cấu hình, một LLM thứ hai rà soát các đề xuất mờ (không trùng tên chính xác) theo lô 20, với các quy tắc về thực thể hành chính Việt Nam, và chuyển những đề xuất đáng ngờ sang duyệt.	LLM

■ Áp dụng gộp

- Đề xuất chỉ được áp dụng khi `apply_mode` của server là `apply`; hiện cài đặt này đang bật. Phép kiểm tra là so sánh nguyên văn, nên mọi giá trị khác đều mặc định an toàn (không áp dụng).
- **Guard hub**: một cặp mà một trong hai node có từ 10 kết nối trở lên không bao giờ được tự động gộp; nó được chuyển sang con người duyệt.
- **Gộp là một giao dịch**: đi theo chuỗi `merged_into` tới node chuẩn đang hoạt động, gộp các alias và nguồn gốc, trở lại cạnh mà không mất mát, đánh dấu node thành viên là superseded, ghi thêm một mục audit hợp nhất và ghi với khóa lạc quan. Mỗi thay đổi được đóng dấu một ID thao tác gộp, nên **tách gộp** khôi phục chính xác trạng thái trước đó.

Chất lượng đã đo (benchmark nội bộ)

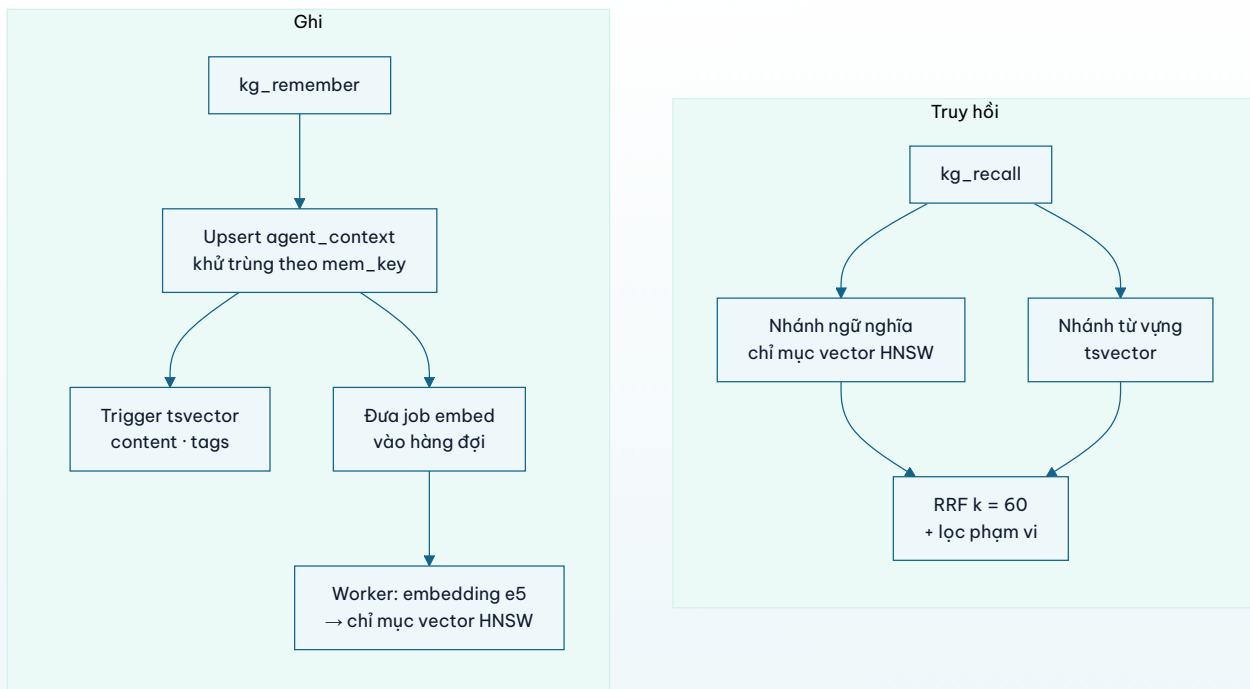
Trước khi bật tự động áp dụng, pipeline ba giai đoạn đã triển khai được kiểm định trên một benchmark tiếng Việt nội bộ quy mô nhỏ (30 nhóm thực thể, 86 cặp trùng lặp; đo vào tháng 6/2026). Kết quả: precision gộp **1.000** và recall **0.915**, không có dương tính giả. Đây là benchmark nhỏ, chuyên biệt theo miền và không nên được hiểu là độ chính xác production trên các kho dữ liệu khác.

■ Bản ghi chủ và bản ghi dẫn xuất

Node chuẩn còn lại sau một lần gộp đóng vai trò **bản ghi chủ** (master record) của thực thể đó (vẫn giữ loại thực thể của nó): mang các alias đã gộp, toàn bộ nguồn gốc truy về từng câu nguồn, và lịch sử audit của mỗi lần gộp. Riêng biệt với việc đó, hệ thống bên ngoài có thể ghi **bản ghi dẫn xuất** (derived record: bản tóm tắt hoặc bản ghi tham chiếu do chính hệ thống đó sở hữu) ngược vào graph một cách idempotent, liên kết chúng tới tài liệu bằng chứng, lấy bản mới nhất (`kg_get_master_record`) và thu hồi chúng.

11 Bộ nhớ của agent BUILT

Agent cần ghi nhớ sở thích, quyết định, dữ kiện và các lần sửa sai qua nhiều phiên làm việc. DAAB cung cấp một kho bộ nhớ nhỏ, bền vững với truy hồi lại, tách khỏi graph chính nhưng nằm trong cùng cơ sở dữ liệu.



Hình 18. Ghi bộ nhớ (embed khi ghi qua hàng đợi job) và truy hồi lại.

- **Loại:** preference, decision, fact, correction. **Phạm vi:** project, user, agent. Bộ nhớ thuộc phạm vi user chỉ được trả về cho chính chủ sở hữu.
- **Ghi idempotent:** một `mem_key` tùy chọn (duy nhất theo dự án, người dùng và phạm vi) biến một lần ghi thành cập nhật, nhờ đó agent có thể tinh chỉnh một ký ức thay vì tích tụ các bản gần trùng.
- **Truy hồi** trộn kết quả ngữ nghĩa và từ vựng bằng cùng Reciprocal Rank Fusion mà tìm kiếm sử dụng.
- **Bảo trì:** một worker lưu giữ chạy nền khử trùng lặp và giới hạn mức tăng trưởng, và một bộ ghi nhận lần truy cập ghi lại thời điểm mỗi ký ức được truy hồi gần nhất.
- **Lịch sử hội thoại:** riêng biệt, agent có thể tìm kiếm toàn văn các luồng hội thoại cũ (`kg_search_sessions`), với mỗi lần đọc được ghi vào nhật ký truy cập.

12 MCP bridge LIVE

Bridge là bề mặt tích hợp của DAAB dành cho AI agent. Nó công bố 55 tool qua Model Context Protocol, trong đó 53 tool đã nối API (hai tool chưa có route phía server), và ánh xạ mỗi lượt gọi tool sang REST API, nên agent nhận đúng cơ chế kiểm tra hợp lệ, phân quyền và audit như dashboard.

• Phương thức truyền tải và triển khai

- **stdio** (mặc định): một client cục bộ duy nhất khởi chạy `kg-bridge serve` như một tiến trình con.

- **Streamable HTTP** (`serve --http`): một endpoint mạng dùng chung, xây trên MCP Go SDK chính thức, với một instance server cho mỗi phiên. Các probe health và readiness nằm ngoài lớp xác thực. Bridge được đóng gói trong cùng image với API server.

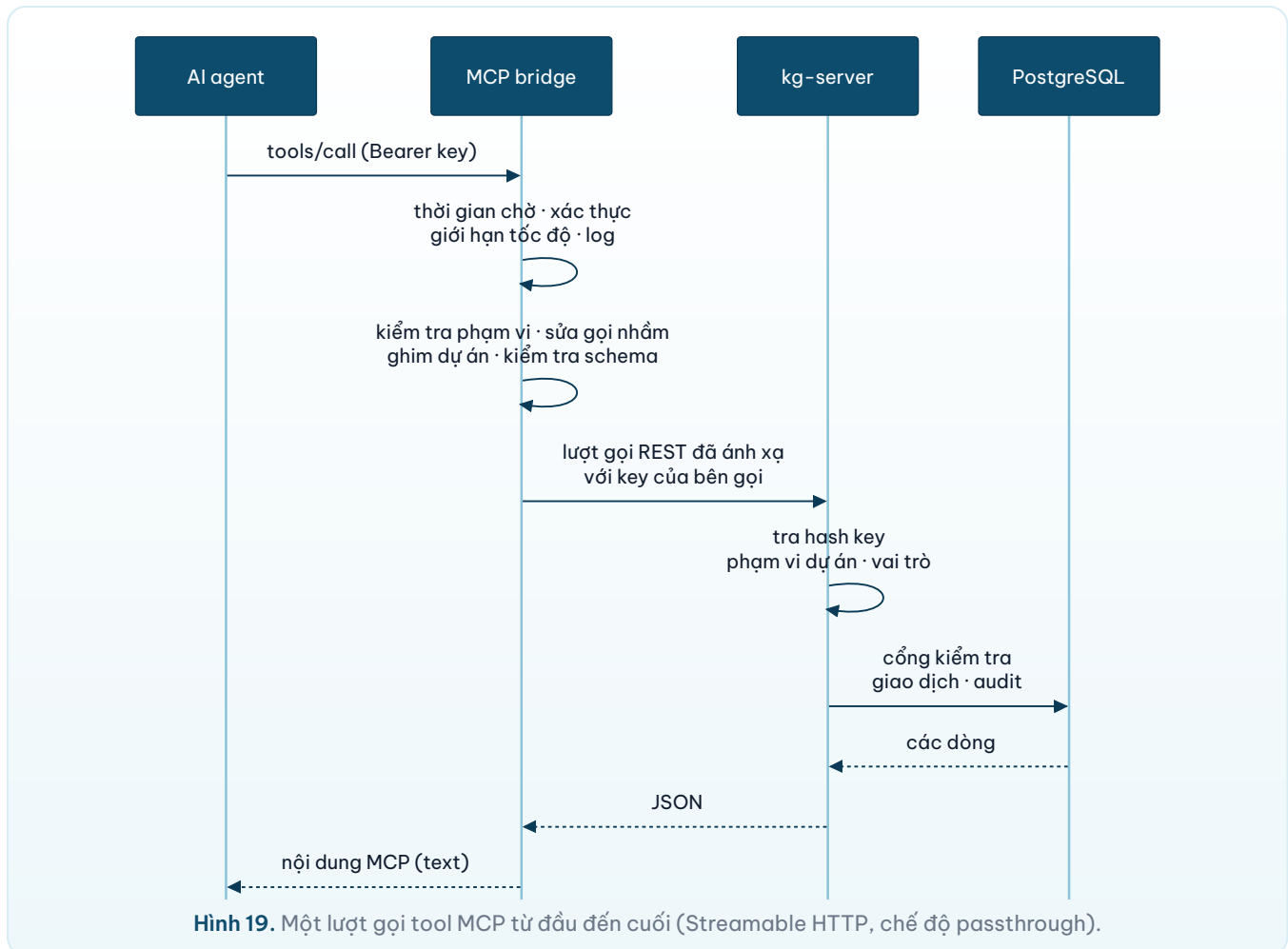
Chế độ xác thực

Chế độ	Hành vi
Passthrough (đa tenant)	Bearer token của mỗi yêu cầu <i>chính là</i> DAAB API key của bên gọi. Bridge dựng một API client theo phiên với key đó, nên server thực thi vai trò và phạm vi dự án của key đó. Một bridge phục vụ nhiều bên sử dụng, mỗi bên được giới hạn trong các dự án mà key của mình cho phép (xem Chương 3 về cách thực thi phạm vi dự án). Key được cấp trên dashboard kèm vai trò và phạm vi dự án.
Gate token (đơn tenant)	Bên gọi xuất trình một bridge token (dài ít nhất 32 byte, hoặc 43 ký tự, kiểm tra khi khởi động); một token thứ hai tùy chọn cấp phạm vi chỉ đọc , trong đó mọi tool thuộc lớp ghi và tool thực thi cục bộ đều bị từ chối. Bridge gọi API bằng service key của chính nó.

Chuỗi middleware HTTP là: phục hồi khi panic → thời gian chờ yêu cầu 60 giây → xác thực → giới hạn tốc độ (120 lượt gọi mỗi 60 giây cho mỗi danh tính) → ghi log lượt gọi → điều phối MCP. Tool thuộc lớp ghi được công bố không kèm annotation MCP `readOnlyHint` , nên client tuân thủ chuẩn sẽ hỏi người dùng trước khi gọi. Một công tắc khởi động tùy chọn khiến bridge từ chối mọi địa chỉ không phải loopback, dành cho các triển khai yêu cầu TLS được kết thúc tại một proxy phía trước.

Pipeline cho mỗi lượt gọi

1. **Kiểm tra phạm vi**: với token chỉ đọc, tool thuộc lớp ghi và thực thi cục bộ bị từ chối.
2. **Sửa gọi nhằm**: các lượt gọi rõ ràng thuộc về tool khác (ví dụ một câu hỏi ngôn ngữ tự nhiên kèm ID nguồn dữ liệu gửi tới tool truy vấn có cấu trúc) được định tuyến lại, hoặc được trả về một lỗi hướng dẫn cho mô hình biết nên dùng tool nào.
3. **Ghim dự án**: khi bridge được cấu hình với một dự án mặc định, nó ghi đè mọi ID dự án do mô hình cung cấp (kèm một mục log). Việc này bảo vệ khỏi trường hợp mô hình gửi nhầm địa chỉ; nó không phải ranh giới tenant. Quyền truy cập do API key trong lượt gọi quyết định, theo cách API thực thi (Chương 3).
4. **Kiểm tra schema** tại bridge (pattern UUID, enum, giới hạn), sau đó ánh xạ sang method HTTP, path, query và body; kết quả được trả về dưới dạng nội dung văn bản MCP.



Ngữ cảnh kết nối. Với mỗi bên gọi, bridge tra cứu các dự án và nguồn dữ liệu mà bên gọi đó truy cập được (cache 5 phút) và trả về chúng trong phần hướng dẫn của MCP server và trong mô tả của các tool chính. Nhờ vậy agent bắt đầu với sẵn các ID cần thiết và không phải tốn lượt gọi để tìm chúng.

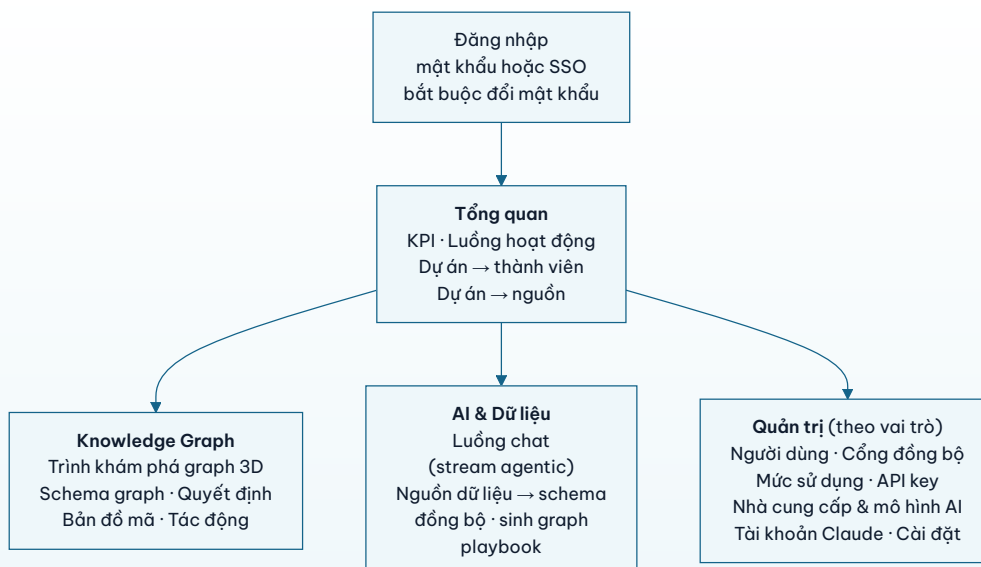
Tệp nằm ngoài giao thức. `kg_request_file_upload` trả về một URL tải lên; client gửi dữ liệu qua HTTP thông thường tới bridge, bridge chuyển tiếp, bằng service key của chính bridge, tới endpoint tải lên của dự án mặc định của bridge. Tài liệu lớn không bao giờ đi dưới dạng thông điệp MCP.

Tool cục bộ. `kg_index_source` / `kg_index_status` chạy code indexer trên host của agent, nơi bridge chạy, với một đường dẫn cục bộ và báo cáo tiến độ. Chúng chỉ hoạt động khi CLI indexer Python được cài trên host đó; image container của bridge không chứa CLI này. Cùng với tool URL tải lên, đây là ba tool duy nhất không dựa trên route REST.

Danh mục tool. 53 tool đã nối API chia thành chín nhóm: ghi nhận tri thức (15), tìm kiếm và điều hướng (9), tài liệu (6), nạp dữ liệu (7), dữ liệu đang vận hành (4), ghi có kiểm soát (3), bộ nhớ và phiên (5), bản ghi (2) và index mã nguồn cục bộ (2); 25 tool thuộc lớp đọc, 25 thuộc lớp ghi và 3 chạy cục bộ. Phụ lục A liệt kê mọi tool kèm mục đích.

13 Web dashboard LIVE

Một ứng dụng Next.js 16 / React 19 phục vụ phía con người của nền tảng: khám phá graph, phân tích dữ liệu dạng hội thoại, kết nối nguồn, duyệt dữ liệu nạp và quản trị quyền truy cập cùng nhà cung cấp AI. Ứng dụng chạy theo mô hình backend-for-frontend (BFF): trình duyệt chỉ giao tiếp với chính origin của dashboard.



Hình 20. Sơ đồ trang của dashboard, nhóm theo thanh điều hướng bên (tổng cộng 33 route trang, gồm trang đăng nhập, các trang chi tiết và các trang nằm ngoài thanh điều hướng).

• BFF proxy và phiên làm việc

- **Đăng nhập** chạy dưới dạng server action: dashboard gọi endpoint đăng nhập (hoặc trao đổi SSO) của API, nhận một API key theo người dùng và lưu vào một cookie phiên được mã hóa, HTTP-only. Một lần đăng nhập mới sẽ thu hồi key dashboard trước đó của người dùng, nên mỗi người dùng chỉ có một phiên dashboard hoạt động.
- **Proxy:** một route catch-all ánh xạ `/api/kg/*` sang `/api/v1/*` của API, gán key của phiên làm bearer token. Server-Sent Events đi qua mà không bị đệm; phản hồi graph được chuẩn hóa về một dạng nodes/edges duy nhất; lỗi 401 từ upstream kết thúc phiên. Một route thứ hai stream các tệp tải lên dạng multipart.
- **Phân quyền** được thực thi trong Go API; dashboard chỉ ẩn mục Quản trị với người không phải admin. Cờ tính năng lấy từ cài đặt công khai của API.
- **Lấy dữ liệu:** trạng thái phía server được quản lý bằng TanStack Query (33 module hook).

• Thời gian thực

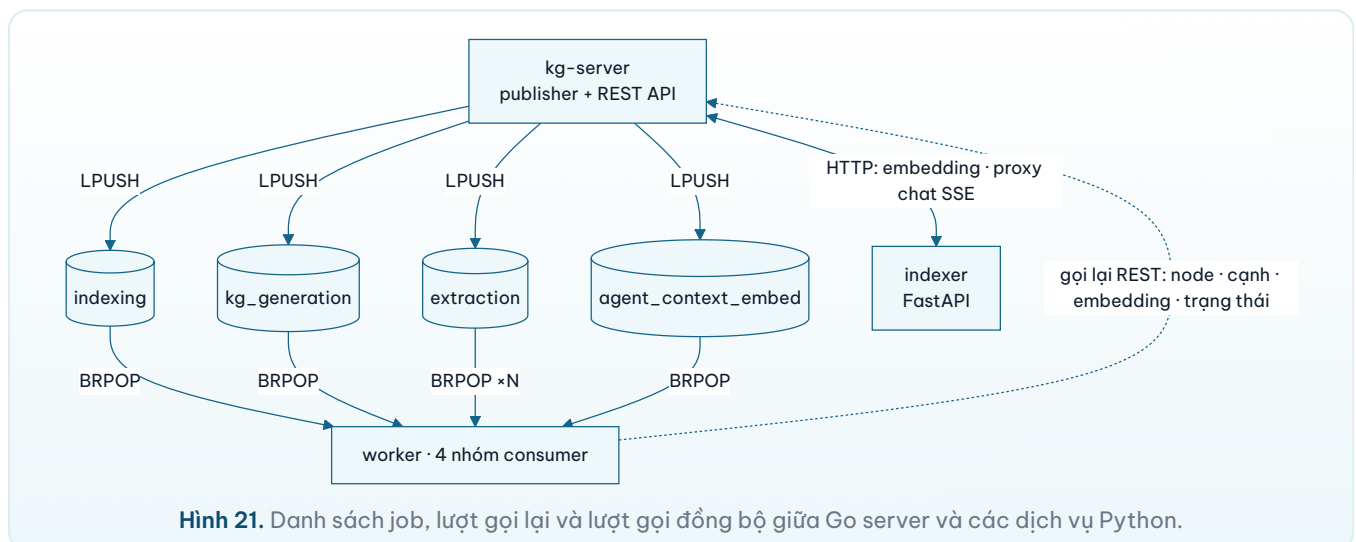
Cơ chế	Được dùng bởi	Chi tiết
Streaming fetch (SSE qua POST)	Chat	Sự kiện có kiểu: tiến độ, token nội dung, khối phong phú, bắt đầu/kết thúc lượt gọi tool, suy luận, yêu cầu làm rõ, tham chiếu node graph, kết quả dữ liệu, hành động gợi ý, hoàn tất. Phía client có thử lại, thời gian chờ giữa các sự kiện và giới hạn số kết nối.
EventSource (SSE qua GET)	Đồng bộ schema và sinh graph	Tiến độ theo từng giai đoạn (trích xuất → sinh node → ánh xạ cạnh → phát hiện quan hệ ngầm → hoàn tất).
Polling thích ứng	Nạp dữ liệu, đồng bộ hệ thống tài liệu, mức sử dụng, bản nháp	Chỉ poll khi job đang ở trạng thái chưa kết thúc và dừng khi job hoàn tất.

Trực quan hóa graph

- **Trình khám phá knowledge graph 3D:** một scene React Three Fiber vẽ toàn bộ node thành một instanced mesh và toàn bộ cạnh thành một tập đường (mỗi loại một draw call), với hiệu ứng bloom và điều khiển quỹ đạo. Bố cục lực (chế độ force, anchor và cluster) chạy trong Web Worker với d3-force-3d và đóng băng khi hội tụ, giữ cho luồng UI rảnh; giao diện được thiết kế cho graph hàng nghìn node.
- **Schema graph:** góc nhìn quan hệ thực thể của bất kỳ cơ sở dữ liệu đã kết nối nào (bảng, khóa ngoại, quan hệ nhiều-nhiều và quan hệ ngầm), render bằng Cytoscape với bố cục dagre.
- **Góc nhìn tác động:** một graph lực 3D của một node được chọn và các láng giềng trực tiếp của nó.

14 Giao tiếp giữa các dịch vụ

Các dịch vụ giao tiếp theo ba cách: job bất đồng bộ trên danh sách Redis, HTTP đồng bộ giữa Go server và Python indexer, và HTTP gọi lại từ worker vào Go API, nơi mọi thao tác ghi vào graph đều đi qua.



Danh sách	Loại thông điệp	Mục đích
indexing	index_project , index_changed	Index toàn bộ hoặc tăng dần mã nguồn của một repository
kg_generation	kg_generation , extract_upload , đồng bộ hệ thống tài liệu	Đưa bản nháp tài liệu đã phê duyệt vào graph; trích xuất tệp đã tải lên; kéo tài liệu từ một hệ thống quản lý tài liệu đã kết nối
extraction	extract_document (một chunk), resolve_document	Trích xuất thực thể mức chunk, sau đó hợp nhất theo tài liệu; được nhiều coroutine đồng thời tiêu thụ
agent_context_embed	embed_agent_context	Embed khi ghi cho bộ nhớ agent

Lượt gọi đồng bộ

- **Go → indexer:** embedding truy vấn cho tìm kiếm ngữ nghĩa và lai, GraphRAG và truy hồi bộ nhớ (thời gian chờ 10 giây); chat streaming, được proxy dưới dạng SSE với tối đa 3 luồng chat đồng thời cho mỗi người dùng.

- **Worker** → **Go**: mọi thao tác ghi graph và báo cáo trạng thái đều dùng lượt gọi REST có xác thực: hoàn tất bản nháp, nội dung và thuộc tính; lô embedding node và bộ nhớ; bản ghi tài liệu chuẩn hóa; tiến độ trích xuất (hoàn tất chunk, đã hợp nhất tài liệu); ứng viên hợp nhất, đề xuất, gộp và tách gộp; yêu cầu liên kết GraphRAG; bản ghi dẫn xuất. Các endpoint dành cho worker nằm dưới cùng lớp xác thực API key như phần còn lại của API `/api/v1`.
- **Go** → **trình duyệt**: luồng tiến độ cho đồng bộ schema, sinh graph và đồng bộ hệ thống tài liệu, được cấp bởi một bộ phát sóng trong tiến trình.

• **Ngữ nghĩa phân phối**

Danh sách hoạt động theo FIFO, và thao tác pop xóa thông điệp, nên việc phân phối là tối đa một lần (at most once). Khả năng chịu lỗi được xây dựng trong chính các job: mỗi handler cô lập lỗi theo từng mục (theo bản nháp, theo tài liệu, theo tập), báo lỗi ngược về API để hiển thị trên dashboard, và không bao giờ đánh dấu hoàn tất một mục đã thất bại. Phía server, một bộ giám sát heartbeat đánh dấu thất bại các job đồng bộ schema không hoạt động quá 30 phút, nên không có gì trông như chạy mãi. Một lượt quét phục hồi phát hành lại các lượt trích xuất bị treo đã được hiện thực nhưng chưa được lên lịch **PLANNED**. Worker tắt một cách êm ái khi nhận SIGTERM.

A Phụ lục A – Danh mục tool MCP

R = lớp đọc (công bố kèm `readOnlyHint`, được phép dùng với token chỉ đọc); W = lớp ghi; L = thực thi cục bộ trên host của agent, nơi bridge chạy. Hai tool khác (một bản tóm tắt ngữ cảnh theo phạm vi và một phân tích tác động) đã được công bố cho client nhưng chưa có route phía server, nên lượt gọi tới chúng sẽ thất bại **PLANNED**; chúng không được tính. Hai tool index lớp L cần code indexer được cài trên host của agent, nơi bridge chạy.

Tool	Lớp	Mục đích
kg_store_decision	W	Tạo node decision (lý do, phương án thay thế), tùy chọn kèm cạnh
kg_store_concept	W	Tạo node concept
kg_store_requirement	W	Tạo node requirement
kg_store_task	W	Tạo node task
kg_store_architecture	W	Tạo node architecture
kg_store_discovery	W	Tạo node discovery
kg_update_decision ... kg_update_discovery (6)	W	Cập nhật từng phần có kiểu cho mỗi loại trong sáu loại node trên
kg_update	W	Cập nhật từng phần dạng tổng quát với khóa lạc quan
kg_link	W	Tạo một cạnh đã kiểm tra theo whitelist
kg_deprecate	W	Đánh dấu node lỗi thời (API node không có thao tác xóa)
kg_search	R	Tìm node toàn văn, ngữ nghĩa hoặc lai
kg_search_chunks	R	Tìm đoạn văn, trả về nguyên văn kèm trích dẫn mục/dòng
kg_query	R	Truy vấn có cấu trúc theo loại node, bộ lọc và số bước tùy chọn
kg_get_node	R	Lấy một node
kg_get_history	R	Lịch sử phiên bản của một node
kg_get_neighbors	R	Láng giềng một bước
kg_traverse	R	Duyệt nhiều bước (tối đa 3 bước)
kg_graph_retrieve	R	Gói chunk xuyên tài liệu của GraphRAG
kg_list_projects	R	Các dự án bên gọi truy cập được
kg_get_document	R	Metadata trích dẫn của tài liệu

Tool	Lớp	Mục đích
kg_get_document_outline	R	Cây mục của một tài liệu
kg_get_section_neighbors	R	Mục cha, mục con và mục anh em của một mục
kg_get_backlinks	R	Tham chiếu tới một node từ các nguồn khác
kg_related_documents	R	Tài liệu có chung thực thể chuẩn, xếp hạng theo độ đặc thù
kg_document_shared_entities	R	Thực thể mà hai tài liệu có chung
kg_ingest_node	W	Gửi một bản nháp tri thức
kg_ingest_batch	W	Gửi tối đa 50 bản nháp
kg_list_drafts	R	Liệt kê bản nháp và trạng thái
kg_approve_drafts	W	Phê duyệt tối đa 50 bản nháp
kg_process_drafts	W	Đưa bản nháp đã phê duyệt vào hàng đợi sinh graph
kg_ingest_status	R	Poll một lần nạp tệp cho tới khi tài liệu sẵn sàng
kg_request_file_upload	L	Lấy URL tải lên theo kênh riêng cho một tệp
kg_list_datasources	R	Tìm các cơ sở dữ liệu đã kết nối
kg_query_datasource	R	Đặt câu hỏi ngôn ngữ tự nhiên với một CSDL đang vận hành (có audit)
kg_query_datasource_status	R	Long-poll lấy kết quả truy vấn (giới hạn số dòng qua <code>max_rows</code>)
kg_describe_table	R	Đặc tả cột của một bảng được phép ghi
kg_insert_datasource_row	W	Chèn một dòng đã bật tường minh, theo whitelist, đã kiểm tra hợp lệ
kg_list_playbooks	R	Các thao tác ghi đã phê duyệt cho một nguồn dữ liệu
kg_execute_playbook	W	Chạy một thao tác ghi nhiều bước đã phê duyệt trong một giao dịch
kg_remember	W	Ghi một ký ức bền vững, có phạm vi
kg_recall	R	Truy hồi bộ nhớ lại
kg_store_session	W	Bắt đầu một phiên làm việc của agent
kg_end_session	W	Kết thúc phiên kèm tóm tắt
kg_search_sessions	R	Tìm kiếm toàn văn lịch sử hội thoại
kg_upsert_derived_record	W	Ghi idempotent một bản ghi do hệ thống bên ngoài sở hữu
kg_get_master_record	R	Bản ghi dẫn xuất mới nhất của một dự án
kg_index_source	L	Index một repository mã nguồn cục bộ vào graph
kg_index_status	L	Poll một job index cục bộ

B Phụ lục B – Tóm tắt REST API

251 route: 247 được đăng ký bởi package handler cộng 4 trên router ngoài cùng (liveness, readiness, version và WebSocket tiến độ đồng bộ). Mọi route `/api/v1/*` đều yêu cầu API key, trừ đăng nhập và trao đổi SSO. Các con số dưới đây là chính xác và cộng lại bằng 251.

Khu vực	Route	Endpoint tiêu biểu
Node, cạnh & tài liệu của graph	38	POST <code>/nodes</code> , tạo/cập nhật có kiểu, PATCH <code>/nodes/{id}</code> , <code>/deprecate</code> , <code>/history</code> , <code>/neighbors</code> , <code>/traverse</code> , <code>/outline</code> , <code>/backlinks</code> , POST <code>/edges</code>
Truy vấn, tìm kiếm & truy xuất	6	<code>/query</code> , <code>/search</code> , <code>/search-chunks</code> , <code>/retrieve/graph</code> , <code>/schema-graph</code>
Nạp dữ liệu: bản nháp, tải lên, kết nối, tài liệu chuẩn hóa, bản ghi dẫn xuất	39	<code>/projects/{id}/ingest[/batch /upload]</code> , <code>/draft-nodes/*</code> , <code>/uploads/*</code> , <code>/connections/*</code> , <code>/canonical-documents/*</code> , <code>/derived-records</code>

Khu vực	Route	Endpoint tiêu biểu
Trích xuất & hợp nhất thực thể (dành cho worker)	17	/ingestion/documents/{id}/extract , /internal/extraction/* , /internal/resolution/* , /internal/graphrag/link , lô embedding node
Nguồn dữ liệu, đồng bộ schema & sinh graph (gồm luồng tiến độ)	28	/data-sources/* , /test-connection , /extract-schema , /sync-schema , /generate-kg , /kg-edges/{id}/confirm reject , /sync/{job}/*
Truy vấn ngôn ngữ tự nhiên, luồng chat & mục yêu thích	25	/ai-queries , /ai-query/stream , /ai/clarification , /threads/* , /favorites/*
Ghi vào CSDL nguồn & playbook	11	/tables/{t}/describe , /insert-row , /playbooks/* (đề xuất, phê duyệt, vô hiệu hóa, thực thi)
Phiên & bộ nhớ agent	10	/sessions/* , /sessions/search , /agent-context/remember recall
Dự án & thành viên	12	/projects/* , lưu trữ/bỏ lưu trữ, thống kê, /index , /members/*
Người dùng, xác thực & API key	25	/users/* , /auth/login supabase logout change-password , OAuth nhà cung cấp, /api-keys/*
Nhà cung cấp AI, mô hình & ngân sách	13	/ai-providers/* (gồm health check, mô hình), /ai-models/* , /ai-functions , /ai-budget-stats
Cài đặt, audit, hoạt động, mức sử dụng & hàng đợi	14	/settings/* , /audit/* , /activity/* , /usage/dashboard , /queue/* , /rate-limit/status
Benchmark BUILT	10	/benchmark/questions runs results (không hiển thị trên điều hướng dashboard)
Health check & phiên bản	3	/healthz , /readyz (ping cơ sở dữ liệu), /api/v1/version
Tổng	251	

Một số vị trí mã nguồn (tương đối theo repository): composition root của API `cmd/kg-server/main.go` ; định tuyến tool MCP `internal/bridge/client.go` ; trộn kết quả lại `internal/store/rrf.go` ; guard thực thi truy vấn `internal/service/source_executor.go` ; GraphRAG `internal/service/graph_retriever.go` ; gộp `internal/service/merge.go` ; pipeline tài liệu `src/ennam_kg/ingestion/pipeline/` ; hợp nhất `src/ennam_kg/resolution/pass2.py` ; BFF proxy `src/app/api/kg/[...path]/route.ts` .